

Project Report
On
Civic Problem - Resolvex

Submitted in the partial fulfillment of the requirement

for the award of degree of

Master of Computer Applications

(Session:2024-26)



Supervised By:

Dr. Ashish Girdhar

Submitted By:

Amit

Roll No. 2024123509

Department of Computer Science and Applications

Kurukshetra University, Kurukshetra – 136119

Declaration by the Candidate

I, Amit, bearing Roll Number 2024123509, hereby declare that the project report titled “ Civic Problem - Resolvex”, submitted in partial fulfillment of the requirements for the award of the Master of Computer Applications of the course M24-CAP-414 , is my original work and has been carried out under the guidance of Dr. Ashish Girdhar.

I also declare that this work has not been submitted previously by me or any other individual for the award of any degree, diploma, or certificate at this or any other institution.

Any references and resources used during the development of this project have been duly acknowledged in the report.

(Candidate's Signature)



Department of Computer Science and Applications
KURUKSHETRA UNIVERSITY KURUKSHETRA
(Established by the State Legislature Act XII of 1956)
(‘A++’ Grade, NAAC Accredited)

Dr. Ashish Girdhar

Email:

Dated: / /2026

Certificate by the Supervisor

This is to certify that the project report titled “Civic Problem - Resolvex” submitted by Amit, Enrollment Number 2024123509, in partial fulfillment of the requirements for the award of the Master of Computer Applications of the course M24-CAP-414, is a bonafide record of original work carried out by him under my supervision during the academic session 2024-26.

This work has not been submitted previously for the award of any degree, diploma, or certificate in this or any other university/institute.

(Supervisor’s Signature)

Dr. Ashish Girdhar

Assistant Professor



Department of Computer Science and Applications
KURUKSHETRA UNIVERSITY KURUKSHETRA
(Established by the State Legislature Act XII of 1956)
(*A++* Grade, NAAC Accredited)

chairperson.dcsa@kuk.ac.in

Prof. Rakesh Kumar

Chairperson

Ext. 2133

Email:

No.CA/26/_____

Dated: / /2026

This is to certify that the project report entitled " Civic Problem - Resolvex" is a bona fide work carried out by Amit, Roll No. 2024123509 student of Master of Computer Applications, under the guidance and supervision of Dr. Ashish Girdhar, Assistant Professor, Department of Computer Science and Applications, Kurukshetra University, Kurukshetra.

(Signature)

Dr. Rakesh Kumar

Chairperson, Department of Computer Science and Applications,
Kurukshetra University, Kurukshetra.

Date:

Seal:

Acknowledgement

Acknowledgement is the perfect way to convey heartiest thanks to all outstanding personalities. I consider myself fortunate to get the opportunity of doing a project, yet the opportunity could not have been utilized without the guidance and support of many individuals.

I wish to express my gratitude to my mentor Dr. Ashish Girdhar, as an Assistant Professor of Department of Computer Science and Applications, Kurukshetra University, Kurukshetra for his cooperation to tell me the right direction in preparing this project. I was fortunate to have him as my project guide. He was ever willing to give me all kind of support and encouragement.

I also wish to express my gratitude to Chairperson Dr. Rakesh Kumar, of Department of Computer Science & Applications, Kurukshetra University, Kurukshetra for his cooperation to tell me the right direction in preparing this project. I was fortunate to have him as my project guide. He was ever willing to give me all kind of support and encouragement.

Finally, if I did not express my strong sense of obligation to my parents and friends, who gave me a supportive atmosphere for working hard.

(Amit)

List of Abbreviation

Abbreviation	Full Form
MERN	MongoDB, Express, React js, Node js
JWT	Json Web Token
API	Application Programming Interface
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
PDF	Portable Document Format
ERD	Entity Relation Diagram
REST	Representational State Transfer
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
DOM	Document Object Model

List of Figures

Fig. No	Figure Name	Page No.
Fig 3.1	Lifecycle of Project	23
Fig 4.1	Architecture Design	28
Fig 4.2	Architecture workflow	29
Fig 4.3	Authentication Flow	31
Fig 4.4	Database Design	31
Fig 7.1	Landing Page	51
Fig 7.2	Register Page	51
Fig 7.3	Login Page	52
Fig 7.4	User Dashboard	52
Fig 7.5 (a)	Raise Complaint	53
Fig 7.5(b)	Location Input	53
Fig 7.5 (c)	Upload File	54
Fig 7.5 (d)	Review before Submission	54
Fig 7.6	My Complaint	55
Fig 7.7	Complaint Status	55
Fig 7.8	User Profile	56
Fig 7.9	Admin Dashboard	57
Fig 7.10	Admin Complaint Management	57
Fig 7.11	Updating the status of the Complaint	58
Fig 7.12	User Management by Admin	58
Fig 7.13	Managing the Department	59
Fig 7.14	Admin Profile	59
Fig 7.15	Complaint Assigned to Dept	60

List of Tables

Table No	Table Name	Page No.
6.1	Register Testcase	45
6.2	Login Testcase	46
6.3	Route Access Testcase	46
6.4	Logout Test Case	47
6.5	Email Verification Testcase	47
6.6	Verify Email Testcase	47
6.7	Complaint Testcase	48
6.8	Complaint by id Testcase	49
6.9	Complaint Status Testcase	49
6.10	Department Assigned Testcase	49
6.11	Admin Dashboard Testcase	50
6.12	Admin Department Testcase	50

Table of Contents

Sr. No.	Title		Page No.
❖	Candidate's Declaration		i
	Certificate from the Supervisor		ii
	Certificate from the Chairperson		iii
	Acknowledgement		iv
	List of Abbreviations		v
	List of Figures		vi
	List of Tables		vii
Chapter 1	Introduction		1-4
	1.1	About the Problem	
	1.2	Introduction to Resolvex	
	1.3	Why Do We Need	
	1.4	Objectives of the Project	
	1.5	Scope of the System	
	1.6	Advantages of the System	
Chapter 2	Tools and Environment Used		5-15
	2.1	Frontend Technologies	
	2.2	Backend Technologies	
	2.3	Development Tools	
Chapter 3	Project Planning and Analysis		16-27
	3.1	Project Overview	
	3.2	Feasibility Analysis	
	3.3	Requirement Analysis	
	3.4	Project Planning	
	3.5	Risk Assessment	
Chapter 4	Project Design		28-32
	4.1	System Architecture Design	
	4.2	Architecture Workflow	
	4.3	Authentication Flow	
	4.4	Database Design	
Chapter 5	Program Code		33-43
	5.1	Backend Code	
	5.2	Frontend Code	
Chapter 6	Testing		44-50
	6.1	Purpose of Testing	
	6.2	Environment for Testing	

	6.3	Authentication Module Testing	
	6.4	Content Generation Testing	
	6.5	User Isolation / Security Testing	
	6.6	Observations and Findings	
Chapter 7	Input and Output Screens		51-60
Chapter 8	Limitation of the Project		61
Chapter 8	Future Scope and Conclusion		62-63
	9.1	Future Scope	
	9.2	Conclusion	
Chapter 10	Bibliography		64

Chapter 1

Introduction

1.1 About the Problem

In many urban areas and towns, people face numerous problems like sanitation, road, electric, drainage, and water complaints, among other civic problems. Although there exist platforms through which citizens can complain about such problems, the methods used to submit the complaints, handle them, and respond are highly flawed.

Some of the existing problems in traditional civic complaint management systems include:

Transparency in complaint handling.

Timeliness in dealing with complaints.

Tracing the process of dealing with complaints.

Such issues not only limit the efficacy of the processes but also discourage citizens from submitting any complaints.

The use of digital transformation is imperative when it comes to solving the above problems. An online system that allows citizens to raise complaints would go a long way in enhancing transparency and efficiency.

1.2 Introduction to Resolvex

Resolvex is an Intelligent Civic Problem Management System implemented using the MERN stack architecture. The application gives citizens the ability to post civic complaints and provides administrators and departments with tools to solve and manage complaints.

The features of Resolvex include:

1. The capability to register complaints along with attaching media files.

2. The ability to suggest complaint category and priority using AI.
3. Citizen, department, and administrator roles, each having different dashboards.
4. Complaint management and notifications to parties involved.
5. SLA Monitoring and analytics.

1.3 Why Do We Need It?

There exist various inefficiencies in the existing civic complaints management system, such as:

- 1. Manual Record Keeping:** The existing complaints management systems have to deal with manual record keeping, making them inefficient.
- 2. Lack of Communication:** There exists no way to keep citizens informed about the progress of their complaints.
- 3. Lack of Accountability:** The lack of tracking systems makes departments accountable for their actions.
- 4. Ineffective Resource Management:** Complaints cannot be prioritized efficiently.
- 5. Lack of Analytical Capabilities:** The lack of central database hampers any analysis.

Resolvex solves such issues.

1.4 Objectives of the Project

The core aims of the Resolvex system are:

- To develop a central digital platform for civic complaints management.
- To enhance communication between the citizens and authorities.
- To make complaint registration and management secure.
- To automatically classify complaints using AI.
- To provide real-time updates on complaints.
- To make processes more transparent and accountable.
- To monitor the performances of various departments using analytical techniques.
- To generate notifications and alerts.

- To provide role-based dashboards.
- To maintain data consistency using MongoDB schemas.

1.5 Scope of the Project

The project scope will include:

Scope of Citizens

- **Secure Registration and Login** are being done by the citizen by filling the name,email and password.
- **Raising Civic Complaints** Complaint can be made by the Citizen for any problem.
- **Image and video uploading** Uploading of the image or video for the Making Complaint.
- **Progress tracking** Tracking the Complaint Status.

Scope of Departments

- **Accepting or rejecting the complaint:** This Accept or reject the Complaint assigned to it.
- **Uploading image to show proof for progress:**Images are Upload for the proof.
- **Assigned Complaint monitoring** monitoring of the complaint is being done at this Panel.

Scope of Admins

- **Department and user management:** He manage the whole user what to do block or delete.
- **Analyzing the system performance:** The System Performance is being analyzed in this.
- **Complaint analysis:** Analysis of the Complaint is being done.

1.6 Advantages of the System

The system has many benefits such as:

- Higher Transparency
- More efficient complaint management
- Higher Accountability
- Centralized management
- Real-time communication
- Reduced paperwork
- AI assisted automation
- Higher citizen participation

Chapter 2

Tools and Technologies Used

2.1 Frontend Technologies

1. React.js

React.js is an open-source JavaScript library that creates user interfaces. React is used in the Resolvex project to generate the frontend part of the application.

The frontend is the part of the system used for interacting with the web applications through various web pages like login form, dashboard, complaint form, analytics page, etc.

React Usage in Resolvex

1. React generates separate UI components that can be reused later.
2. Based on role dashboards are created using react.
3. The complaint forms can be generated dynamically using React.
4. The real-time updates are displayed instantly using React.
5. The APIs can be linked via Axios.
6. Routing of pages is done effectively.

Benefits of Using React in the Project

1. Enhanced user experience
2. Reusable code
3. Single component structure
4. Easier maintenance
5. Efficient navigation

2. Vite

It is an up-to-date front end build tool, which enables quick and effective development and optimized production build process.

Usage in Resolvex

1. Vite is used in Resolvex for Frontend Server
2. Frontend loading speed optimization is done by the vite in the resolvex.
3. Building optimized production files vite plays a very important in it.

Advantages

1. Extremely fast start-up time
2. Easy development process
3. Instant changes while coding
4. Lightweight configurations

4. Tailwind CSS

It is a utility-first CSS framework, used to develop responsive interfaces.

Usage in Resolvex

Tailwind CSS is used for designing ,Login Page, Complaint Form, Admin Dashboard, Cards

Tables, Buttons and Notifications.

Benefits

1. Easy UI development
2. Excellent Design
3. Responsiveness
4. Minimum CSS coding
5. A modern design

4. React Router DOM

React Router DOM manages all types of navigation in frontend pages.

Usage in Resolvex

React Router DOM provides:

- Navigation without refreshing the page

- Implementing Protected routes
- Implementing role-based routing

Example of Routes

- **Public Routes:** These are the routes of the resolvex that can be accessed by anyone without any protected routes. These are the open routes for everyone wheter it is of any role user.
 - **/login:** this routes is being accessed for the login page visiting.
 - **/signup:** This Routes is for going to the Register page of resolvex to register new user.
- **Citizen Routes**
 - **/dashboard:** This routes is being made for going to the dashboard of the the citizen where it will get the different details such as the complaint made by him or many features also.
 - **/raise-complaint:** On this routes the citizen will file the complaint for the complaint he want to make it can be of any type he can mark complaint in the 3 steps only.
- **Admin Routes**
 - **/admin/dashboard:** This routes is for the admin dashboard as all the details were shown to the admin for all the user amd complaint and the departments also.
- **Department Routes**
 - **/dept/assigned:** On this routes the Department user can see the assigned complaint to him.

Benefits

- Effective navigation are made by the React router dom very effective and best for good and fast navigation.
- Easier user experience make the experience of the user very easy

- Protection of routes the protected or public routes are being maintain by this React router.

5. Axios

Axios is a library for managing requests between frontend and backend.

Usage in Resolvex

The following requests are made using Axios

1. Login request: The request from this is being sent to the backend server for the data verification of the user and the login.
2. Complaint creation request: The complaint creation also being maintain by this Axios in the resolvex.
3. Fetch complaint request: all the Complaint from the database from all the user are being fetched by the axios request.

Example Request Types

- GET Request:
Fetching complaint from backend, fetch the users or department from the backend.
- POST Request:
Sending new complaint information,creating of new user and so on.
- PUT Request:
Updating the complaint status from the admin end or from the department end.

Benefits

- Easy handling of APIs
- Asynchronous requests
- Secure communication

6. Recharts

Recharts is a charting library used for displaying the analytics in dashboard

Usage in Resolvex

1. Displaying: the report for the all the data such as for the sla.
2. Complaint Analytics: Analysing the Complaints for all the which are solved or not and many more.
3. Department Performance Analytics: Performance of the department is also been managed in this how many complaint is assigned to the department and how many are solved.
4. SLA Analytics: This is also been shown by the Rechats.

Benefits

- Interactive Charts
- Better analytics
- Data representation

2.2 Backend Technologies

1. Node.js:

In Resolvex, Node.js serves as the backend runtime environment for executing JavaScript on the server. Node.js is used in various ways, such as executing backend APIs, running authentication logic, performing real-time socket communication, and processing file uploads. There are many reasons for why Node.js is used on the project, including fast execution time, event-driven architecture, non-blocking operations, and scalable architecture.

2. Express.js:

Express.js is a framework used on the backend for creating REST APIs.

Express.js is used for creating backend routes, such as :

1. /api/auth

2. /api/complaints
3. /api/admin

The benefits of using Express.js include being:

- lightweight framework
- easy routing; middleware support and scalable architecture on the backend.

3. MongoDB:

It is a No sql based database store the data in the key value format.

MongoDB is the primary database used for Resolvex.

MongoDB is used to store data for:

Users.

Complaints.

Notifications.

Departments.

Comments.

There are many reasons for the selection of MongoDB, including its:

- flexible schema.
- JSON-like documents.
- easy to scale.

4. Mongoose:

Mongoose is an object-data mapping (ODM) library used for MongoDB.

Mongoose is used for:

- **Creating schemas:** Schema are created by the mongoose such as userSchema, complaintSchema.
- **Validating data:** For validating the data we used the mongoose for better data storage in the database.

- **Defining models:** For defining the model which keys or field should be present in the database collections.

The example models used in the project are:

User, complaint, department and notification.

The benefits of using Mongoose include:

schema validation, cleaner database code, and better querying capabilities.

5. JWT Authentication

Json Web Token for giving the security to the request made from the user it is attached with the each request made by the user.

JWT is used for secure authentication.

How JWT Works in Resolvex:

Step 1: Login

User enters credentials.

Step 2: Token Generation

Server generates JWT token.

Step 3: Token Storage

Frontend stores token in localStorage.

Step 4: Protected Requests

Token attached to API requests.

Step 5: Verification

Backend verifies token.

Benefits

- Secure authentication
- Stateless sessions
- Faster access control

- Better scalability

6. Bcryptjs

It convert the password in the hashing format for the password security storing.

bcryptjs provides security for user passwords.

How bcryptjs is Used

- When a user creates an account or changes their password, bcryptjs will:
- Store hashed passwords
- Use salt rounds

This makes it impossible for even system administrators to view the user's password in plaintext.

Benefits

- Strong password security
- Protects against plaintext password storage
- Helps prevent password-based attacks

7. Multer

All the images or video which are stored during the complaint registering are being handled by the multer.

Multer handles uploads of files using the multipart/form-data encoding type.

How Multer Works

End users can upload:

1. Images of complaints are being stored using this in the encoding format.
2. Videos of complaints are stored in the database using the multer .

Benefits

- Secure file uploads
- Efficient file handling

- Support for multiple files at once

8. Cloudinary

Cloudinary allows users to store uploaded files in the cloud.

How Cloudinary Works

After a file is uploaded:

1. The uploaded file is sent to multer.
2. The uploaded file is then sent to Cloudinary.
3. Cloudinary returns the URL of the uploaded file.
4. The URL is stored in MongoDB.

Benefits

- Files stored in the cloud
- Files can be delivered more quickly
- Files managed securely

9. Socket.io

It handles all the real time tasks of the System to manage it and update the data.

Socket.io enables real-time communication between the frontend and backend of the application.

How Socket.io Works

- When the status of a complaint changes:
- The backend emits an event to notify frontend users.
- Front-end users receive updates about the complaint status immediately.
- Users see the updated complaint status without needing to refresh their pages.

Types of Real-time Features

- Notifications
- Updated status of complaints

Benefits

- Users communicate with each other and the system instantly
- Improved usability for users
- Real-time synchronization

10. Nodemailer

Nodemailer allows for sending emails from back-end systems.

Nodemailer Utilisation

Nodemailer will send email from a system to notify users of the following events:

- Email Verification

Nodemailer Benefits

- Automate communications
- Offer secure verification pathways
- Improve overall user security

11. Gemini AI

Application of Artificial Intelligence (AI) in the project has enabled intelligent capabilities.

Application of AI

- Complaint Classification
- AI predicts the:
 - Complaint type
 - Priority of complaint
- Complaint Suggestions
- AI will allow a user to interact with the system using a chatbot.

AI Benefits

- Reduce human involvement with the classification of complaints
- Increase overall efficiency

- Decrease the time taken to process a complaint

2.3 Development Tools:

- **VS Code** – Used for coding and development.
- **Postman** – Used for API testing.
- **Git & GitHub** – Used for version control and project management.
- **ChatGPT and DeepSeek:** Used for error resolving and logic improvement

Chapter 3

Project Planning & Analysis

3.1 Project Overview:

The Resolvex is a Civic Problem Management System that operates on a current stack of MERN technologies designed to create a digital complaint management system efficiently both for the customer, departments, and administrators.

The system has three primary levels of design – front end, back end, and database.

1. Front End: The front-end allows users to interact with the system via a responsive dashboard and several pages Built on react & tailwind CSS.

2. Back End: The back-end includes all business logic & provides authentication, handles workflows for complaints, has AI integrated, processes API calls, and is built on node.js and express.js.

3. Database: The database layer is based on MongoDB which stores user data (personally identifiable information), complaint data, notification information, comment history, and department information.

3.2 Feasibility Analysis:

Feasibility analysis helps establish the viability of the project; it assesses whether it can technically and economically be executed and what the benefits of successful implementation are.

➤ Technical Feasibility:

The Resolvex project can be considered to be technically feasible, as the MERN stack provides all of the technologies necessary for building scalable and secure web applications.

Some of the Technical advantages of using the MERN technologies include:

- Complete JavaScript-based development;
- REST API's that are scalable;
- Reusable, adaptable front-end components;
- NoSQL database structure that makes them adaptable;
- Real-time communication link between front end and back end;
- Enhanced cloud-based media storage;
- The Ability to Integrate AI Systems.

The Resolvex project integrates the following technologies:

- Front-end created in react;
- Its back-end built with express;
- The database built using MongoDB;
- JSON web tokens for authentication;
- Socket.IO for real-time updates from sockets;
- Gemini for AI services;
- Cloudinary for media storage.

All of these technologies are recognized as modern and are industry-standard; each one has very good support from the community and provides great benefits to users.

➤ **Operational feasibility:**

The system will be easy to use for citizens and departments, thus speeding up the complaint handling process for departments, providing centralised monitoring for administrators, real-time communication and notifications, reducing the amount of paperwork that must be completed manually, and improving accountability. The workflow is consistent with existing municipal processes for dealing with complaints.

➤ **Economic feasibility:**

The project will be developed using open source technologies and free development tools, allowing the implementation costs to be very low. Technologies used for the

project will include React, Node.js, Express.js, MongoDB, Tailwind CSS, and Socket.io, and because no expensive proprietary software will be needed to develop the project, it is an economical option for both academic and practical purposes.

➤ **Time Feasibility:**

Modular development will allow the project to be developed in a way that is manageable within academic timelines. The development modules include:

Authentication

Complaint management

Department workflow

Real-time communication

Notification

Each module can be developed and tested separately, making development much easier than if they had to be developed and tested together.

3.3 Requirement Analysis

Requirement analysis consists of

the complete listing of both functional and non-functional requirements for the system.

➤ **Functional requirements:**

❖ **Authentication Module:** The system must provide secure login and user management for all users. Functions of the authentication module will include:-

- **User registration:** The user will register successfully is being maintain in the Authentication module by verifying the email id of the user and the role assigned to it.

- **Login/logout:** The login logout was also authenticated in this when the user login the token generate for the authentication and while the user logout token expire.
- **Generation of JWT tokens:** It is generated for the Verifying the user request for valid user.
- **Email Verification:** Users are able to verify their emails upon registration to ensure that only authorized users have access to the platform. It is aimed at improving the security of user accounts.
- **Authorization Based on Roles:** Users will be assigned different levels of access depending on their roles such as Citizen, Department User, and Admin.
- **Management of User Profiles:** Users are provided with a way of managing their profiles by viewing or updating their profile details such as name, contact information, and profile picture.

❖ **Citizen Complaint Management Citizens**

- **Complaint Submission:** Citizens may create and submit complaints that pertain to civic matters via an online form. The system will document all relevant information such as the complaint title, description, location, and its category.
- **Image and Video Submission:** Images and videos can be attached by the users along with complaint submission. This will provide departments with a clear picture of what is going on.
- **View Past Complaint History:** Users are allowed to access a record of past complaints with status and other information available for each one. It will be easier for citizens to access their records of the past.

❖ **Department Management Module:**

- **View Assigned Complaints:** A department member is able to view complaints assigned to his/her department. This facilitates better departmental management of workload.
- **Accepting or Rejecting Complaints:** Department members are able to check and accept complaints deemed genuine or reject those which are duplicate or invalid.
- **Updating the Complaint's Status:** Department heads are able to update the complaint's status. For example, the updated statuses may include 'in progress,' 'pending' or 'resolved'.
- **Uploading Evidence of Completion of Complaints:** A department member may be able to upload proof in form of image or documents for a complaint which has been sorted out.
- **Mark the Complaint as Completed:** If the problem has been sorted out successfully, it will be marked as being completed by the department member.

❖ Admin Management Module

- **Manage Users:** Admins can manage their users by creating accounts, updating, blocking, and deleting accounts. This ensures that there is an orderly administration of the users.
- **Create Departments:** The admin can form different departments according to the civic services that each department caters to. This helps in efficient management of complaints.
- **Assign Complaints:** The admins have the ability to manually allocate complaints to various departments whenever necessary. This will ensure that the complaint is allocated to the right department.
- **Generate SLA Reports:** The admins can generate the SLA report. It will help the administrators know how effectively the complaints have been resolved within the deadline.

- **Block and Unblock Users:** The admins will be able to block and unblock users whenever there is need to do so. It will ensure security and orderliness of the system.
- **Monitor Overall Activity Throughout the System:** The administrator will be able to supervise the activities happening throughout the system.

➤ **Non-Functional Requirements (NFR)**

❖ **Security**

- **Validation of JWT Tokens:** JWT tokens will help authenticate user session. It prevents anyone from accessing sensitive information on APIs and protected routes.
- **Encryption of Password Using bcrypt :**User passwords should be hashed using the bcrypt method before being saved into the database.
- **API Protection:** Important routes of the APIs should be secured against any unauthorized access. Middleware validation allows the exchange of information between client and server safely.
- **Access Based on RBAC:** Access control based on RBAC will protect sensitive operations from unauthorized actions by unprivileged users.

❖ **Scalability**

- **Large Set of Complaints:** The app needs to deal with large amounts of complaint records and their accompanying multimedia files well. Proper database optimization increases scalability.
- **Modular Design Structure:** The application should use a modular architecture design in which its components can be easily handled separately. It will improve reusability and increase productivity.

❖ Reliability

- **Consistency of the Database:** It is important that all database entries stay consistent. Correctly handling data will prevent any loss or distortion of information.
- **Exceptional Error Handling:** The application needs to offer efficient error messaging to users. It will improve system stability and usability.
- **Stable API Responses:** The APIs of the application should provide stable responses to requests made by its users. Reliable communication will ensure efficient operation of the frontend and backend.
- **Validated Data Methods:** All entered data needs to go through validation to avoid errors. It will decrease the number of invalid entries in the system.

❖ Performance

- **Fast API Response Times:** There needs to be quick responses from APIs used by the app to ensure good performance. The backend must be efficient.
- **Optimal Front End Page Load & Render Time:** The frontend should be loaded and rendered quickly in all devices and browsers used by the users. This will offer a seamless experience.
- **Optimized Database Operations:** Efforts should be made to optimize database operations by minimizing unnecessary processing times. Effective querying can help accomplish this.

❖ Usability

- **Responsive Design:** The platform should effectively adapt to different screen sizes including those of desktop computers, tablets and mobile phones.

- **Good Navigation Capability:** There should not be difficulty navigating through different features and pages of the platform. A clear menu can help.
- **Effortless Complaint Submission:** The platform should make it effortless for complaints to be submitted. This will encourage citizenry to lodge complaints efficiently.

3.4 Project Planning

The project development was divided into multiple phases to ensure systematic implementation and proper module integration.

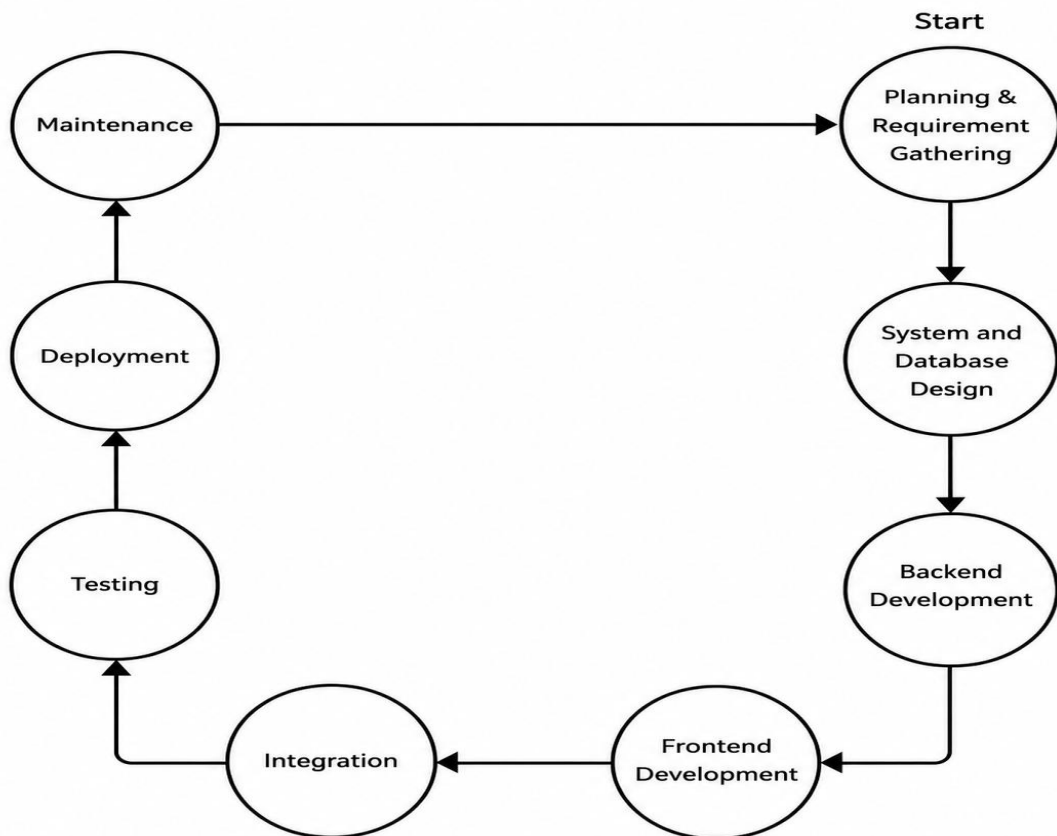


Fig 3.1 Lifecycle of Project

Phase I: Gathering Requirements

During the requirements gathering phase, the workflows of civic complaints and end-user requirements were identified.

Activities:

- Identifying the system users
- Understanding the Civic Complaint Workflows
- Identifying the required Modules

Phase II: Design System and Design Database

The design system and database phase focus on architectural planning of software architecture and creation of the database schema.

Activities:

- Designing the MERN Architecture
- Creating MongoDB Schemas
- Planning API structure

Phase III: Development of the Backend

Implementation of the backend APIs and business logic using Node.js/Express.js.

Features Developed:

- Login/Authentication API
- Complaints API
- Admin API
- Department API

Phase IV: Development of the Frontend

Frontend Interfaces and Dashboard created using React and Tailwind CSS.

Pages Created:

- Login/Register Page
- Citizen Dashboard

- Admin Dashboard
- Department Dashboard
- Complaint Page

Phase V: Integration

Integration of the Frontend and Backend Modules or interfaces.

Tasks Completed:

- Integration of the Axios API
- Integration of the JWT Authentication Flow
- Integration of the Real-Time Socket

Phase VI: Testing

Testing and debugging through Manual Testing.

Types of Testing Completed:

- Testing Authentication
- Testing the Citizen Complaint Workflow
- Testing Uploads
- Testing APIs
- Testing Role Authorization

Phase VII: Deployment Preparation

Preparing the Application for Production Deployment.

Tasks Completed:

- Environmental Variables
- Security Checks
- Improvements to Error Handling

3.5 Risk Assessment

Assessment of risk pertains to the evaluation of possible problems and the methods for minimizing risk.

➤ Risk Assistant with Security:

- **Possible Security Risks:**

- Theft of JWT Token
- Unauthorized access to JWT Token
- Weak Password Handling
- Misuse of API

- **Methods for Minimizing Security Risks:**

- Verification of JWT Token through Middleware
- Password Hashing with Bcrypt
- Role-Based Authorization
- Routes that are Protected

➤ Risk Related to Data Integrity

- **Possible Risks to Data Integrity:**

- Wrong Status Update of Complaint
- Duplicate Entry of Complaint
- Invalid Record in Database.

- **Methods for Minimizing Risk to Data Integrity:**

- Validation of Complaint by Mongoose
- Control of Status Change
- Validation of Complaint by Backend Logic

➤ Risk Assistant with File Uploading

- **Possible Risks Assistant with File Uploading:**

- Uploading of Malicious Software by Users
- Uploading Unsupported File Types
- File Size exceeding Maximum Allowed Limits

- **Methods for Minimizing File Uploading Risk:**
 - Filter Uploading Files using Multer
 - Limit the Size of All Uploaded Files
 - Validate the MIME Type of All Uploaded Files
 - Store all Uploaded Files Using Cloudinary

➤ **Risk Assistant with Scalability**

- **Possible Risks Related to Scalability:**
 - Maximum Size of Complaint Database
 - Increased Load on the API
 - Large Number of Concurrent Users Accessing the API
- **Methods for Minimizing Scalability Risk:**
 - Optimize Queries to MongoDB
 - Use Modular Architecture
 - Use Efficient API Handling

➤ **Risk Related to Testing**

- **Possible Risks Related to Testing:**
 - No Automation for Testing
 - Failure to Identify Bugs
 - Inconsistent Workflows
- **Methods for Minimizing Testing Risk:**
 - Conduct Manual Testing of API
 - Validate User Interface
 - Perform Testing at Route Level
 - Conduct Testing for Authentication

Chapter 4

Project Design

4.1 System Architecture

The system follows MERN architecture:

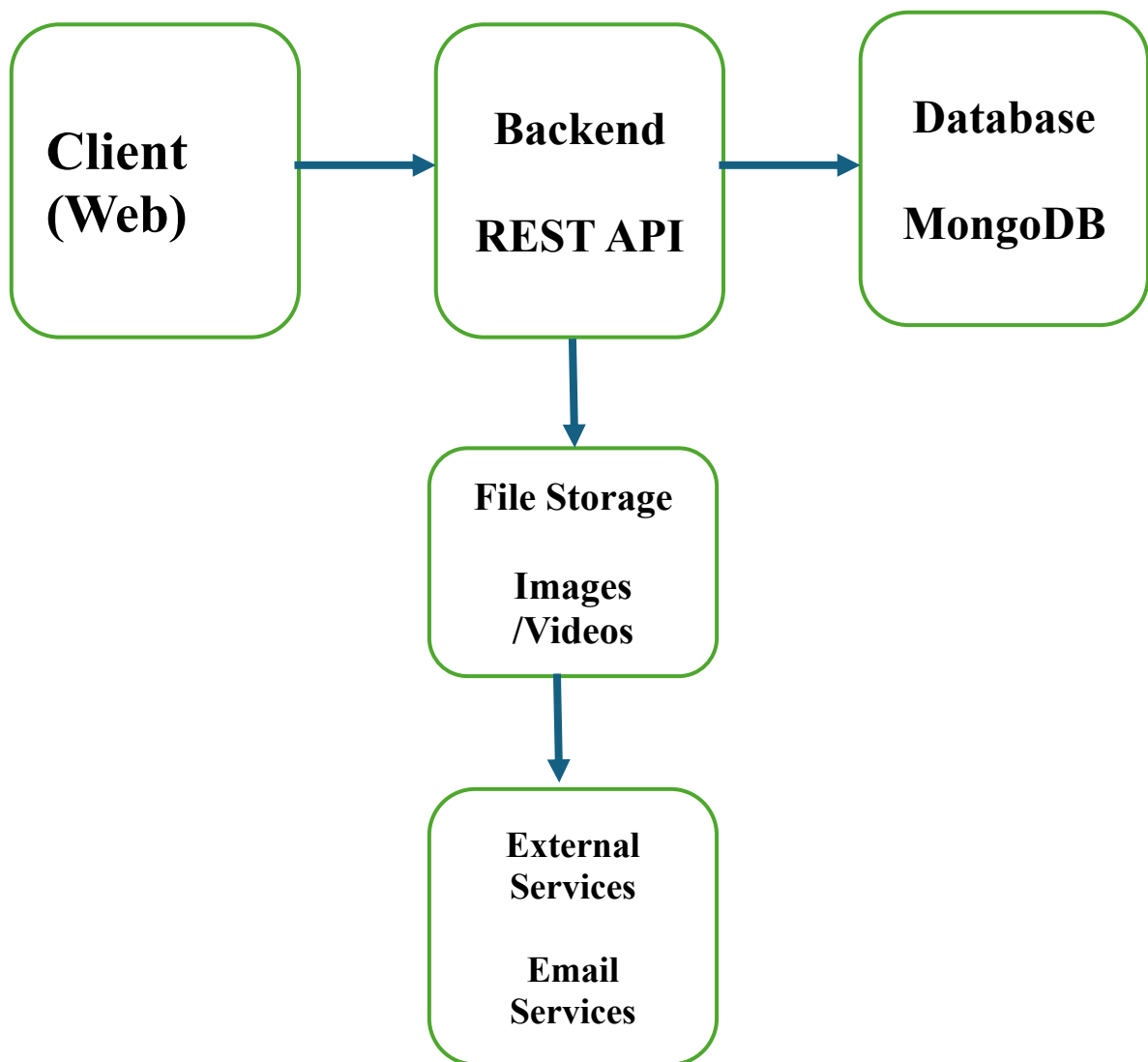


Fig 4.1 Architecture Design

4.2 Architecture Workflow

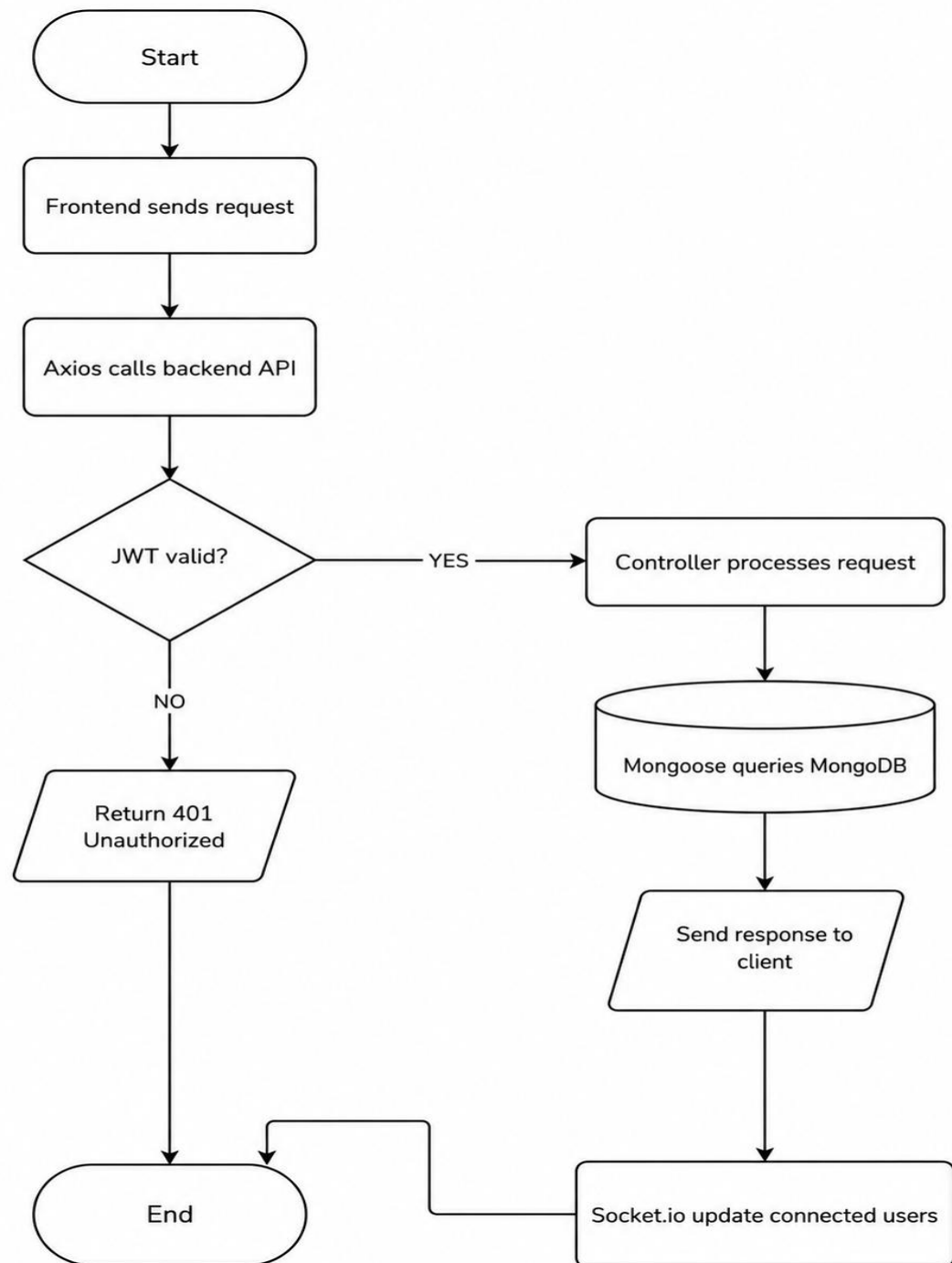


Fig 4.2 Flowchart

ser sends request from frontend.

1. Axios communicates with backend API.
2. Express validates JWT.
3. Controller processes request.
4. Mongoose interacts with MongoDB.
5. Response sent back to client.
6. Socket events update connected users.

4.3 Authentication Flow

Registration

- User submits signup form.
- Password hashed using bcrypt.
- User stored in MongoDB.
- Verification email sent.

Login

- Credentials verified.
- JWT generated.
- Token sent to client.

Protected Access

- Middleware validates token.
- Role authorization applied.

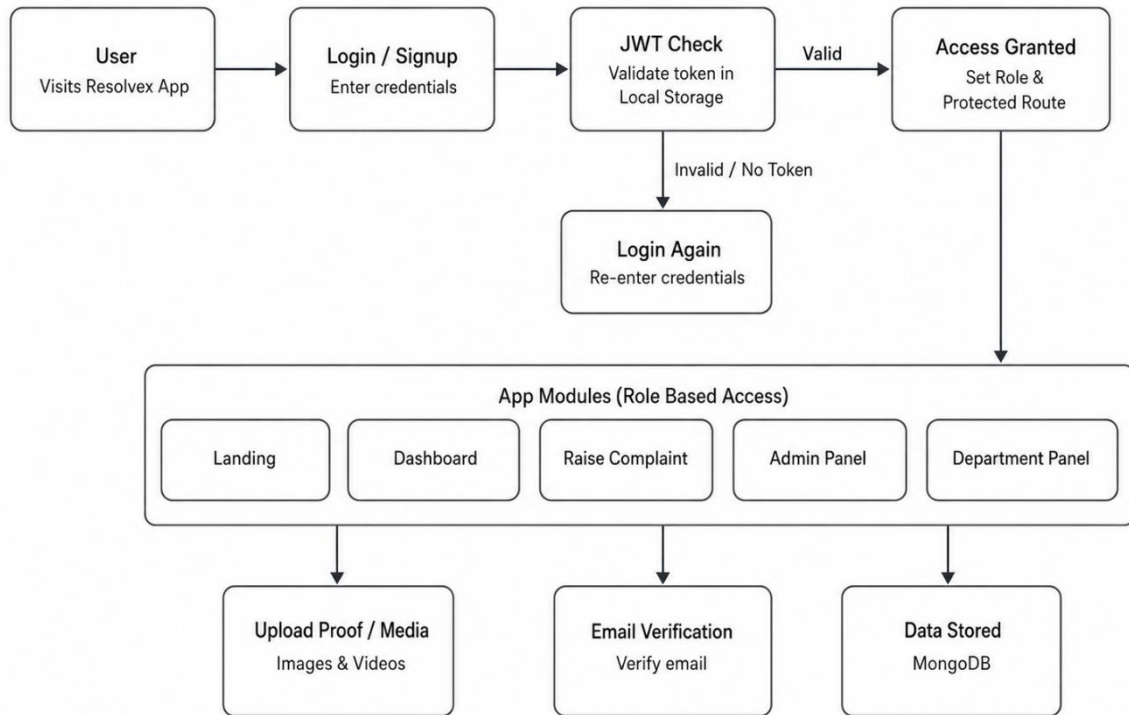


Fig 4.3 Authentication Flow

4.4 Database Design

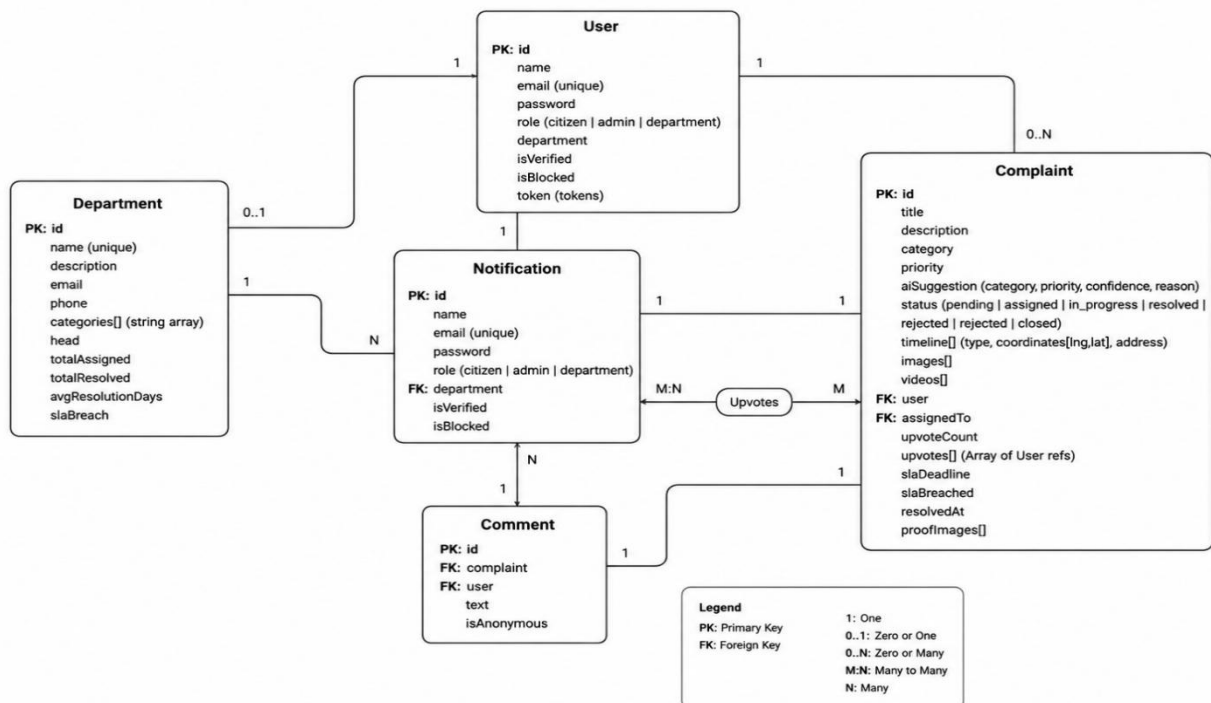


Fig 4.4 Database Design

1. User Collection

It is like a table in the RDBMS which store the data in it for the user.

It Stores:

Name : name of the citizen

Email: Email of the User

Password: Password of the User

And etc.

2. Complaint Collection

The Complaint collection store the all the complaint related field in it to perform the operations on it.

It Stores:

Title: Title of the report in String Format

Description: Details Description of the Complaint

Status: Current Status of the report

Department: Which department it is assigned

3. Department Collection

The Department Collection store the department collection it may include the:

Department name, Email ,Description, head and etc.

Stores department information.

4. Notification Collection

The notification store all the notification data in it has the user,message and etc.

Stores user notifications.

5. Comment Collection

The Comment Collection store the Comment data who comment , what is commented, when it is commented all this is store in the Comment collection.

Chapter 5

Program Code

5.1 Backend Code

Sercver/Server.js

```
const server = http.createServer(app);
const io = socketIo(server, {
  cors: { origin: process.env.CLIENT_URL || 'http://localhost:5173',
    methods: ['GET', 'POST'] },
});
socketService.initSocket(io);
// Connecting the database
connectDB();

// middleware
app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
app.use(cors({ origin: process.env.CLIENT_URL || 'http://localhost:5173', credentials: true }));
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));

app.use('/uploads', express.static(path.join(__dirname, '../uploads')));

const limiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 200,
  message: 'Too many requests, please try again later.' });
const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 20 });
app.use('/api/', limiter);
app.use('/api/auth/', authLimiter);
app.use('/api/auth', authRoutes);
app.use('/api/complaints', complaintRoutes);
app.use('/api/admin', adminRoutes);
app.use('/api/departments', deptRoutes);
app.use('/api/notifications', notifRoutes);
app.use('/api/ai', aiRoutes);
app.get('/api/health', (req, res) => res.json({ success: true,
  message: 'Resolvex API is running', timestamp: new Date() }));
cron.schedule('0 * * * *', async () => {
  try {
    const Complaint = require('../models/Complaint');
    await Complaint.updateMany(
      { status: { $nin: ['resolved', 'rejected'] }, slaDeadline:
        { $lt: new Date() }, slaBreached: false },
      { slaBreached: true }
    );
    console.log('🔔 SLA breach check completed');
  } catch (err) {
    console.error('SLA cron error:', err.message);
  }
});
app.use((err, req, res, next) => {
  console.error(err.stack);
```

```

    res.status(err.status || 500).json({ success: false, message:
err.message || 'Internal Server Error' });
});
app.use((req, res) => res.status(404).json({ success: false, message:
'Route not found.' }));
const PORT = process.env.PORT || 5000;
server.listen(PORT, () => console.log(`🚀 Resolvex server running on
port ${PORT} in ${process.env.NODE_ENV} mode`));

```

server/controllers/auth.js

```

const crypto = require('crypto');
const User = require('../models/User');
const asyncHandler = require('express-async-handler');
const { signToken } = require('../middleware/auth');
const emailService = require('../services/emailService');
const sendTokenResponse = (user, statusCode, res) => {
  const token = signToken(user._id);
  res.status(statusCode).json({ success: true, token, user });
};

exports.register = asyncHandler(async (req, res) => {
  const { name, email, password, role } = req.body;
  const existingUser = await User.findOne({ email });
  if (existingUser) {
    return res.status(400).json({ success: false, message: 'Email
already registered.' });
  }
  const allowedRole = role === 'citizen' ? 'citizen' : 'citizen';
  const verifyToken = crypto.randomBytes(32).toString('hex');
  const user = await User.create({
    name, email, password, role:
allowedRole, emailVerifyToken: verifyToken,
    emailVerifyExpire: Date.now() + 24 * 60 * 60 * 1000, // 24h
  });
  try {
    await emailService.sendVerificationEmail(email, verifyToken);
  } catch (e) {
    console.error('Email send failed:', e.message);
  }

  sendTokenResponse(user, 201, res);
});

exports.login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ success: false, message: 'Please
provide email and password.' });
  }
  const user = await User.findOne({ email }).select('+password');
  if (!user || !user.password) {
    return res.status(401).json({ success: false, message: 'Invalid
credentials.' });
  }
  const isMatch = await user.comparePassword(password);
  if (!isMatch) {
    return res.status(401).json({ success: false, message: 'Invalid
credentials.' });
  }
  if (user.isBlocked) {

```

```

    return res.status(403).json({ success: false, message: 'Your
account has been blocked. Contact support.' });
  }
  if (!user.isVerified) {
    return res.status(403).json({
      success: false,
      message: 'Please verify your email before logging in.',
      code: 'EMAIL_NOT_VERIFIED',
      email: user.email
    });
  }
  user.lastLogin = Date.now();
  await user.save({ validateBeforeSave: false });

  sendTokenResponse(user, 200, res);
});

exports.verifyEmail = asyncHandler(async (req, res) => {
  const { token } = req.params;
  const user = await User.findOne({
    emailVerifyToken: token,
    emailVerifyExpire: { $gt: Date.now() },
  });

  if (!user) {
    return res.status(400).json({ success: false, message: 'Invalid
or expired verification link.' });
  }
  user.isVerified = true;
  user.emailVerifyToken = null;
  user.emailVerifyExpire = null;
  await user.save({ validateBeforeSave: false });
  res.json({ success: true, message: 'Email verified successfully.
You can now login.' });
});

exports.forgotPassword = asyncHandler(async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user) {
    return res.status(404).json({ success: false, message: 'No user
with that email.' });
  }

  const resetToken = crypto.randomBytes(32).toString('hex');
  user.resetPasswordToken = resetToken;
  user.resetPasswordExpire = Date.now() + 60 * 60 * 1000; // 1h
  await user.save({ validateBeforeSave: false });

  await emailService.sendPasswordResetEmail(user.email, resetToken);
  res.json({ success: true, message: 'Password reset email sent.' });
});

exports.resetPassword = asyncHandler(async (req, res) => {
  const user = await User.findOne({
    resetPasswordToken: req.params.token,
    resetPasswordExpire: { $gt: Date.now() },
  });

  if (!user) {
    return res.status(400).json({ success: false, message: 'Invalid
or expired reset link.' });
  }

```

```

    user.password = req.body.password;
    user.resetPasswordToken = null;
    user.resetPasswordExpire = null;
    await user.save();
    res.json({ success: true, message: 'Password reset successful.
Please login.' });
});
exports.getMe = asyncHandler(async (req, res) => {
    const user = await
User.findById(req.user._id).populate('department', 'name');
    res.json({ success: true, user });
});
exports.updateProfile = asyncHandler(async (req, res) => {
    const { name, phone, address, notificationPrefs, avatar } =
req.body;
    const update = {};
    if (name !== undefined) update.name = name;
    if (phone !== undefined) update.phone = phone;
    if (address !== undefined) update.address = address;
    if (avatar !== undefined) update.avatar = avatar;
    if (notificationPrefs !== undefined) update.notificationPrefs =
notificationPrefs;
    const user = await User.findByIdAndUpdate(req.user._id, update, {
        new: true,
        runValidators: true,
    }).populate('department', 'name');
    res.json({ success: true, user });
});
exports.changePassword = asyncHandler(async (req, res) => {
    const { currentPassword, newPassword } = req.body;
    const user = await User.findById(req.user._id).select('+password');
    const isMatch = await user.comparePassword(currentPassword);
    if (!isMatch) return res.status(400).json({ success: false,
message: 'Current password is incorrect.' });
    user.password = newPassword;
    await user.save();
    res.json({ success: true, message: 'Password changed
successfully.' });
});

```

Server/controllers/complaintController.js

```

exports.createComplaint = asyncHandler(async (req, res) => {
    const { title, description, isAnonymous, address } = req.body;
    let { lat, lng } = req.body;
    lat = parseFloat(lat) || 0;
    lng = parseFloat(lng) || 0;
    const images = (req.files?.images || []).map((f) => ({
        url: `/uploads/${f.filename}`,
        publicId: f.filename,
    }));
    const videos = (req.files?.videos || []).map((f) => ({
        url: `/uploads/${f.filename}`,
        publicId: f.filename,
    }));
    const slaDeadline = new Date();
    slaDeadline.setDate(slaDeadline.getDate() + (SLA_DAYS[ai.priority]
|| 7));
    const complaint = await Complaint.create({

```

```

        title,description,category: ai.category,priority: ai.priority,
        aiSuggestion: { category: ai.category, priority: ai.priority,
confidence: ai.confidence, reason: ai.reason },
        isSpam: ai.isSpam,
        spamReason: ai.spamReason,
        location: { type: 'Point', coordinates: [lng, lat], address:
address || '' },
        images,
        videos,
        user: req.user._id,
        isAnonymous: !!isAnonymous,
        status: 'pending',
        slaDeadline,
        timeline: [{ status: 'pending', message: 'Complaint submitted
successfully.', updatedBy: req.user._id, updatedAt: new Date() }],
    });
    const populated = await complaint.populate('user', 'name email
avatar');
    socketService.alertAdmin({ complaint: populated, message: 'New
complaint received!' });
    res.status(201).json({ success: true, complaint: populated,
aiSuggestion: ai });
});
exports.getComplaints = asyncHandler(async (req, res) => {
    const { status, category, priority, search, page = 1, limit = 10,
sort = '-createdAt' } = req.query;
    const filter = {};
    if (req.user.role === 'citizen') filter.user = req.user._id;
    else if (req.user.role === 'department') filter.assignedTo =
req.user.department;
    if (status) filter.status = status;
    if (category) filter.category = category;
    if (priority) filter.priority = priority;
    if (search) filter.$or = [{ title: { $regex: search, $options:
'i' } }, { description: { $regex: search, $options: 'i' } }];
    const skip = (page - 1) * limit;
    const [complaints, total] = await Promise.all([
        Complaint.find(filter).sort(sort).skip(skip).limit(Number(limit))
        .populate('user', 'name avatar').populate('assignedTo', 'name'),
        Complaint.countDocuments(filter),
    ]);

    res.json({ success: true, complaints, total, page: Number(page),
pages: Math.ceil(total / limit) });
});
exports.getComplaint = asyncHandler(async (req, res) => {
    const complaint = await Complaint.findById(req.params.id)
        .populate('user', 'name avatar email')
        .populate('assignedTo', 'name email phone')
        .populate('timeline.updatedBy', 'name role');

    if (!complaint) return res.status(404).json({ success: false,
message: 'Complaint not found.' });

    const comments = await Comment.find({ complaint: complaint._id })
        .populate('user', 'name avatar role')
        .sort('createdAt');

    res.json({ success: true, complaint, comments });

```

```

});
exports.upvoteComplaint = asyncHandler(async (req, res) => {
  const complaint = await Complaint.findById(req.params.id);
  if (!complaint) return res.status(404).json({ success: false,
message: 'Complaint not found.' });
  const userId = req.user._id.toString();
  const alreadyUpvoted = complaint.upvotes.map((u) =>
u.toString()).includes(userId);
  if (alreadyUpvoted) {
    complaint.upvotes = complaint.upvotes.filter((u) =>
u.toString() !== userId);
    complaint.upvoteCount = Math.max(0, complaint.upvoteCount - 1);
  } else {
    complaint.upvotes.push(req.user._id);
    complaint.upvoteCount += 1;
  }
  await complaint.save();
  res.json({ success: true, upvoted: !alreadyUpvoted, upvoteCount:
complaint.upvoteCount });
});
exports.addComment = asyncHandler(async (req, res) => {
  const { text, isAnonymous } = req.body;
  const complaint = await Complaint.findById(req.params.id);
  if (!complaint) return res.status(404).json({ success: false,
message: 'Complaint not found.' });
  const comment = await Comment.create({ complaint: req.params.id,
user: req.user._id, text, isAnonymous: !!isAnonymous });
  const populated = await comment.populate('user', 'name avatar
role');
  if (complaint.user.toString() !== req.user._id.toString()) {
    await socketService.createNotification({
      userId: complaint.user,
      title: 'New Comment',
      message: `Someone commented on your complaint
"${complaint.title}".`,
      type: 'comment',
      complaintId: complaint._id,
    });
  }
  res.status(201).json({ success: true, comment: populated });
});
exports.getNearby = asyncHandler(async (req, res) => {
  const { lat, lng, radius = 5000 } = req.query;
  if (!lat || !lng) return res.status(400).json({ success: false,
message: 'lat and lng required.' });
  const complaints = await Complaint.find({
    location: {
      $near: {
        $geometry: { type: 'Point', coordinates: [parseFloat(lng),
parseFloat(lat)] },
        $maxDistance: Number(radius),
      },
    },
    status: { $ne: 'resolved' },
  }).limit(50)
  .select('title category priority status location upvoteCount
createdAt');

  res.json({ success: true, complaints });
});
exports.getHeatmap = asyncHandler(async (req, res) => {

```



```

    const complaints = await Complaint.find({ 'location.coordinates':
    { $ne: [0, 0] } })
      .select('location category priority status')
      .limit(500);
    const points = complaints.map((c) => ({
      lat: c.location.coordinates[1],
      lng: c.location.coordinates[0],
      category: c.category,
      priority: c.priority,
      status: c.status,
    }));
    res.json({ success: true, points });
  });

```

5.2 Frontend Code

App.jsx

```

const addTimeline = (compint, status, message, userId) => {
  complaint.timeline.push({ status, message, updatedBy: userId,
updatedAt: new Date() });
};

exports.createComplaint = asyncHandler(async (req, res) => {
  const { title, description, isAnonymous, address } = req.body;
  let { lat, lng } = req.body;
  lat = parseFloat(lat) || 0;
  lng = parseFloat(lng) || 0;
  const ai = await aiService.classifyComplaint(title, description);
  const images = (req.files?.images || []).map((f) => ({
    url: `/uploads/${f.filename}`,
    publicId: f.filename,
  }));
  const videos = (req.files?.videos || []).map((f) => ({
    url: `/uploads/${f.filename}`,
    publicId: f.filename,
  }));
  const slaDeadline = new Date();
  slaDeadline.setDate(slaDeadline.getDate() + (SLA_DAYS[ai.priority]
|| 7));
  const complaint = await Complaint.create({
    title,
    description,
    category: ai.category,
    priority: ai.priority,
    aiSuggestion: { category: ai.category, priority: ai.priority,
confidence: ai.confidence, reason: ai.reason },
    isSpam: ai.isSpam,
    spamReason: ai.spamReason,
    location: { type: 'Point', coordinates: [lng, lat], address:
address || '' },
    images,
    videos,
    user: req.user._id,
    isAnonymous: !!isAnonymous,
    status: 'pending',
    slaDeadline,
    timeline: [{ status: 'pending', message: 'Complaint submitted
successfully.', updatedBy: req.user._id, updatedAt: new Date() }],

```

```

    });
    const populated = await complaint.populate('user', 'name email
avatar');
    socketService.alertAdmin({ complaint: populated, message: 'New
complaint received!' });
    res.status(201).json({ success: true, complaint: populated,
aiSuggestion: ai });
});
exports.getComplaints = asyncHandler(async (req, res) => {
    const { status, category, priority, search, page = 1, limit = 10,
sort = '-createdAt' } = req.query;

    if (req.user.role === 'citizen') filter.user = req.user._id;
    else if (req.user.role === 'department') filter.assignedTo =
req.user.department;

    if (status) filter.status = status;
    if (category) filter.category = category;
    if (priority) filter.priority = priority;
    if (search) filter.$or = [{ title: { $regex: search, $options:
'i' } }, { description: { $regex: search, $options: 'i' } }];

    const skip = (page - 1) * limit;
    const [complaints, total] = await Promise.all([
        Complaint.find(filter).sort(sort).skip(skip).limit(Number(limit))
        .populate('user', 'name avatar').populate('assignedTo', 'name'),
        Complaint.countDocuments(filter),
    ]);
    res.json({ success: true, complaints, total, page: Number(page),
pages: Math.ceil(total / limit) });
});
exports.updateStatus = asyncHandler(async (req, res) => {
    const { status, remarks } = req.body;
    const complaint = await
Complaint.findById(req.params.id).populate('user', 'name email
notificationPrefs');
    if (!complaint) return res.status(404).json({ success: false,
message: 'Complaint not found.' });
    complaint.status = status;
    if (remarks) complaint.departmentRemarks = remarks;
    if (status === 'resolved') complaint.resolvedAt = new Date();
    res.json({ success: true, complaint });
});
exports.assignComplaint = asyncHandler(async (req, res) => {
    const { departmentId } = req.body;
    const dept = await Department.findById(departmentId);
    if (!dept) return res.status(404).json({ success: false, message:
'Department not found.' });
    Complaint.findById(req.params.id).populate('user', 'name email');
    if (!complaint) return res.status(404).json({ success: false,
message: 'Complaint not found.' });

    complaint.assignedTo = departmentId;
    complaint.status = 'assigned';
    complaint.assignedAt = new Date();
    addTimeline(complaint, 'assigned', `Assigned to ${dept.name}`,
req.user._id);
    await complaint.save();
    dept.totalAssigned += 1;

```

```

    await dept.save();

    // Notification for the dept users
    const deptUsers = await User.find({ department: departmentId, role:
'department' });
    for (const du of deptUsers) {
        await socketService.createNotification({
            userId: du._id,
            title: 'New Complaint Assigned',
            message: `Complaint "${complaint.title}" has been assigned to
your department.`,
            type: 'assignment',
            complaintId: complaint._id,
        });
    }
    res.json({ success: true, complaint });
});
exports.upvoteComplaint = asyncHandler(async (req, res) => {
    const complaint = await Complaint.findById(req.params.id);
    if (!complaint) return res.status(404).json({ success: false,
message: 'Complaint not found.' });

    const userId = req.user._id.toString();
    const alreadyUpvoted = complaint.upvotes.map((u) =>
u.toString()).includes(userId);

    if (alreadyUpvoted) {
        complaint.upvotes = complaint.upvotes.filter((u) =>
u.toString() !== userId);
        complaint.upvoteCount = Math.max(0, complaint.upvoteCount - 1);
    } else {
        complaint.upvotes.push(req.user._id);
        complaint.upvoteCount += 1;
    }

    await complaint.save();
    res.json({ success: true, upvoted: !alreadyUpvoted, upvoteCount:
complaint.upvoteCount });
});
exports.addComment = asyncHandler(async (req, res) => {
    const { text, isAnonymous } = req.body;
    const complaint = await Complaint.findById(req.params.id);
    if (!complaint) return res.status(404).json({ success: false,
message: 'Complaint not found.' });

    const comment = await Comment.create({ complaint: req.params.id,
user: req.user._id, text, isAnonymous: !!isAnonymous });
    const populated = await comment.populate('user', 'name avatar
role');

    if (complaint.user.toString() !== req.user._id.toString()) {
        await socketService.createNotification({
            userId: complaint.user,
            title: 'New Comment',
            message: `Someone commented on your complaint
"${complaint.title}".`,
            type: 'comment',
            complaintId: complaint._id,

```

```

    });
  }
  res.status(201).json({ success: true, comment: populated });
});
exports.getNearby = asyncHandler(async (req, res) => {
  const { lat, lng, radius = 5000 } = req.query;
  if (!lat || !lng) return res.status(400).json({ success: false,
message: 'lat and lng required.' });
  const complaints = await Complaint.find({
    location: {
      $near: {
        $geometry: { type: 'Point', coordinates: [parseFloat(lng),
parseFloat(lat)] },
        $maxDistance: Number(radius),
      },
    },
    createdAt
  });
  res.json({ success: true, complaints });
});
exports.getHeatmap = asyncHandler(async (req, res) => {
  const { lat: [0, 0] } = req.query;
  .select('location category priority status')
  .limit(500);
  const points = complaints.map((c) => ({
    lat: c.location.coordinates[1],
    lng: c.location.coordinates[0],
    category: c.category,
    priority: c.priority,
    status: c.status,
  }));
  res.json({ success: true, points });
});

```

Landing.jsx

```

export default function Landing() {
  return (
    <div className="landing-container">
      <nav className="landing-nav">
        <div className="landing-logo fade-in-up">
          
          <span>Resolvevex</span>
        </div>
        <div className="landing-nav-links fade-in-up">
          <Link to="/login" className="btn btn-ghost">Login</Link>
          <Link to="/signup" className="btn btn-primary">Sign
Up</Link>
        </div>
      </nav>
      <main className="landing-hero">
        <div className="hero-badge fade-in-up">
          <Zap size={16} /> Empowering Citizens, Transforming Cities
        </div>
        <h1 className="hero-title fade-in-up"
style={{ animationDelay: '0.1s' }}>
          Civic Issues Resolved <br />
          <span className="hero-title-
highlight">Intelligently.</span>
        </h1>
      </main>
    </div>
  );
}

```

```

        <p className="hero-subtitle fade-in-up"
style={{ animationDelay: '0.2s' }}>
            Resolvev leverages AI to classify, route, and resolve
municipal complaints in record time. Report issues, track progress,
and build a better community together.
        </p>

        <div className="hero-actions fade-in-up"
style={{ animationDelay: '0.3s' }}>
            <Link to="/signup" className="btn btn-primary btn-lg">
                Report an Issue <ArrowRight size={18} />
            </Link>
            <Link to="/login" className="btn btn-ghost btn-lg">
                Track Existing Complaint
            </Link>
        </div>
        <div className="features-grid fade-in-up"
style={{ animationDelay: '0.4s' }}>
            <div className="feature-card">
                <div className="feature-icon-wrapper">
                    <Activity size={24} />
                </div>
                <h3 className="feature-title">AI-Powered Routing</h3>
                <p className="feature-desc">Smart algorithms
automatically classify and assign your complaints to the right
department instantly.</p>
            </div>
            <div className="feature-card">
                <div className="feature-icon-wrapper">
                    <MapPin size={24} />
                </div>
                <h3 className="feature-title">Precise Geolocation</h3>
                <p className="feature-desc">Pinpoint issues exactly where
happen. Live map integration ensures workers know exactly where to
go.</p>
            </div>
            <div className="feature-card">
                <div className="feature-icon-wrapper">
                    <Shield size={24} />
                </div>
                <h3 className="feature-title">Transparent Tracking</h3>
                <p className="feature-desc">Real-time status updates from
submission to resolution. Total transparency into municipal
operations.</p>
            </div>
        </div>
        </main>
        <footer className="landing-footer">
            <p>© {new Date().getFullYear()} Resolvev. All rights
reserved.</p>
        </footer>
    </div>
    );
}

```

Chapter 6

Testing

6.1 Purpose of Testing

The Purpose of Testing is to Validate Expected Behaviors of Software Under All Possible Conditions, Including Edge Cases. Testing Provides Confirmation That the Software Operates Securely and as Intended.

Goals of Testing Include:

- Detecting Software Bugs
- Validating Software Functionality
- Ensuring Security of the Software
- Improving Reliability of the Software

6.2 Environment for Testing

It is Composed of the Following:

- Frontend (UI): localhost:5173
- Backend (API): localhost:5000
- Database: Local MongoDB Instance

The Types of Testing Tools that Will Be Used Are:

- Web Browser Testing
- Postman for Testing APIs

6.3 Authentication Module Testing

6.3.1 Test Cases – Register (POST /api/auth/register)

Table 6.1 Register Testcase

TC	Description	Input	Expected Output	Actual Output	Status
1	Valid Citizen Registration	Name:ABC, email:abc@gmail.com, password:12345678,	User Register Successfully	201, User Created	Pass
2	Registering with duplicate Email	Email:abc@gmail.com	Email Already Registered	400, Email Registered	Pass
3	Empty Field	Field is Empty	Fill the Required Field	500/400, Fill all the field	Pass
4	Invalid Email Format	Email: abcgmail.com	Enter Valid Email	500/400, Invalid Email	Pass
5	Password & Confirm Password not match	Password:12345 Confirm:123456	Password not Match	Password not Match	Pass

6.3.2 Test Cases – Login(POST /api/auth/login)

Table 6.2 Login Testcase

TC	Description	Input	Expected Output	Actual Output	Status
1	Valid Login	Email,Password	200, Return JWT token	200, Return JWT token	Pass
2	Invalid Password	Correct Email, Wrong Password	401, Invalid email or Password	401, Invalid email or Password	Pass
3	Invalid Email	Correct Email, Wrong Password	401, Invalid email or Password	401, Invalid email or Password	Pass
4	Empty Input	Blank Field	401, Fill the credentials	401, Fill The Credential	Pass
5	Valid User but Unverified	Valid email and password	403, Email not verified	403, Email not verified	Pass

6.3.3 Test Cases – Protected Route Access(GET /api/auth/me)

Table 6.3 Route Access Testcase

TC	Description	Input	Expected Output	Actual Output	Status
1	Access with valid JWT cookie	Cookie present and valid	200 OK, user data returned	200, correct user	Pass

2	Access without any cookie	No cookie sent	401 "Not authorized. Please log in."	401, correct message	Pass
3	Access with tampered/invalid JWT	Malformed token string	401 "Invalid token. Please log in again."	401, JsonWebTokenError caught	Pass
4	Access with expired JWT	Manually expired token	401 "Token expired. Please log in again."	401, TokenExpiredError caught	Pass

6.3.4 Test Cases - Logout (POST /api/auth/logout)

Table 6.4 Logout Testcase

TC	Description	Expected Output	Actual Output	Status
1	Logout clears cookie	200 OK, cookie cleared (maxAge = 0)	200, Set-Cookie header with empty token	Pass

6.3.5 Test Cases - Verification (POST /api/auth/resend-verification)

Table 6.5 Email Verification Testcase

TC	Description	Input	Expected Output	Actual Output	Status
1	Unverified user exists	Valid email	200/201; new token stored	200, Set-Cookie header with empty token	Pass
2	Verified user	Valid email	200/403 depending on implementation	200/403 depending on implementation	Pass

6.3.6 Test Cases - Verification (GET /api/auth/verify-email/:token)

Table 6.6 Verify Email Testcase

TC	Description	Input	Expected Output	Actual Output	Status
----	-------------	-------	-----------------	---------------	--------

1	Valid or unexpired token	Valid token	200 Verified	200 Verified	Pass
2	Expired or invalid token	invalid token	400/404 Not Found	400/404 Not Found	Pass

6.4 Content Generation Testing

Dashboard analytics testing:

- Complaint Register
- Complaint Status
- Assigned Department

Citizen filtering testing:

- Ensure users only access personal records

6.4.1 Test Cases – Registering Complaint (POST /api/complaints)

Table 6.7 Complaint Testcase

TC	Description	Input	Expected Output	Actual Output	Status
1	Required Field Filled	Title,description Filled, No media	201, Complaint status set to pending	201, Complaint status set to pending	Pass
2	Without title and Description	Blank title and description	403, Fill title and Description	403, Fill title and Description	Pass
3	Images Uplaod between 1 to 5 And video 1,2	Image upload 1...5 in images or video 1,2	201; images saved and video	201; images saved & video saved	Pass
4	Images greater than limit >5 and video >2	Images more than 5 or video more than 2	400 multer limit	400 multer limit	Pass

6.4.2 Test Cases – Getting Complaint (GET /api/complaints/:id)

Table 6.8 Complaint by Id Testcase

TC	Description	Input	Expected Output	Actual Output	Status
1	Check for Complaint	Valid input	200; status updated	200; status updated	Pass
2	Check for Complaint	Valid Input	Update status 200	Update status 200	Pass

6.4.3 Test Cases – Updating Complaint (PUT /api/complaints/:id/status)

Table 6.9 Complaint Status Testcase

TC	Description	Input	Role	Expected Output	Actual Output	Status
1	Update Complaint	Valid Complaint Id	Admin	200; complaint returned	200; complaint returned	Pass
2	Update Complaint	InValid Complaint Id	Department	400/404 Not Found	400/404 Not Found	Pass
3	Update Complaint	NA	Citizen	403, Unauthorized User	403, Unauthorized User	Pass

6.4.4 Test Cases – Assigned Department (GET /api/department/assigned)

Table 5.10 Department Assigned Testcase

TC	Description	Input	Role	Expected Output	Actual Output	Status
1	Update Complaint	Valid Complaint Id	Admin	200; complaint returned	200; complaint returned	Pass
2	Update Complaint	InValid Complaint Id	Department	400/404 Not Found	400/404 Not Found	Pass
3	Update Complaint	NA	Citizen	403, Unauthorized User	403, Unauthorized User	Pass

6.4.5 Test Cases – Admin Dashboard (GET /api/admin/dashboard)

Table 6.11 Admin Dashboard Testcase

TC	Description	Input	Role	Expected Output	Actual Output	Status
1	Citizen try to access Admin	NA	Citizen	403, Unauthorized Access	403, Unauthorized Access	Pass
2	Admin Login	Valid admin	Admin	200,Dashboard	200,Dashboard	Pass

6.4.6 Test Cases – Department Manage by Admin (/api/admin/departments)

Table 6.12 Admin Department Testcase

TC	Description	Input	Expected Output	Actual Output	Status
1	Department Creation	Valid Input	201; created; unique name	201; created; unique name	Pass
2	Get All Dept.	NA	200,List	200,List	Pass
3	Update Department	Enter Details	200,Updated	200,Updated	Pass

6.5 User Isolation and Security Assessment

The objective of Security Assessment is to provide security for:

- Protects Citizen Data
- Proper Role Restrictions
- Endpoints Protected

6.6 Observations/Findings

Observations:

- JWT Authentication works properly
- Role-Based Authorization structurally working
- Able to Execute CRUD Functionalities

Limitations:

- No AI Integration
- No Rate Limiting

Chapter 7

Input Output Screen

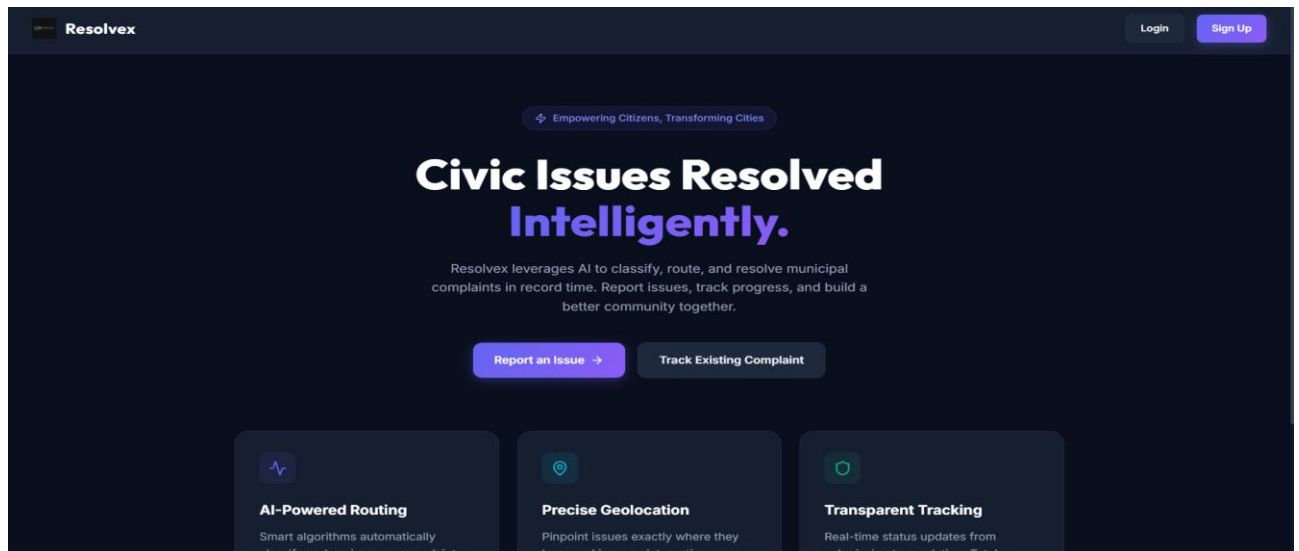


Fig 7.1 Landing Page

- This is the First Interaction page of the User where he can login or register the citizen from the navbar.
- Some features about the project is shown here.

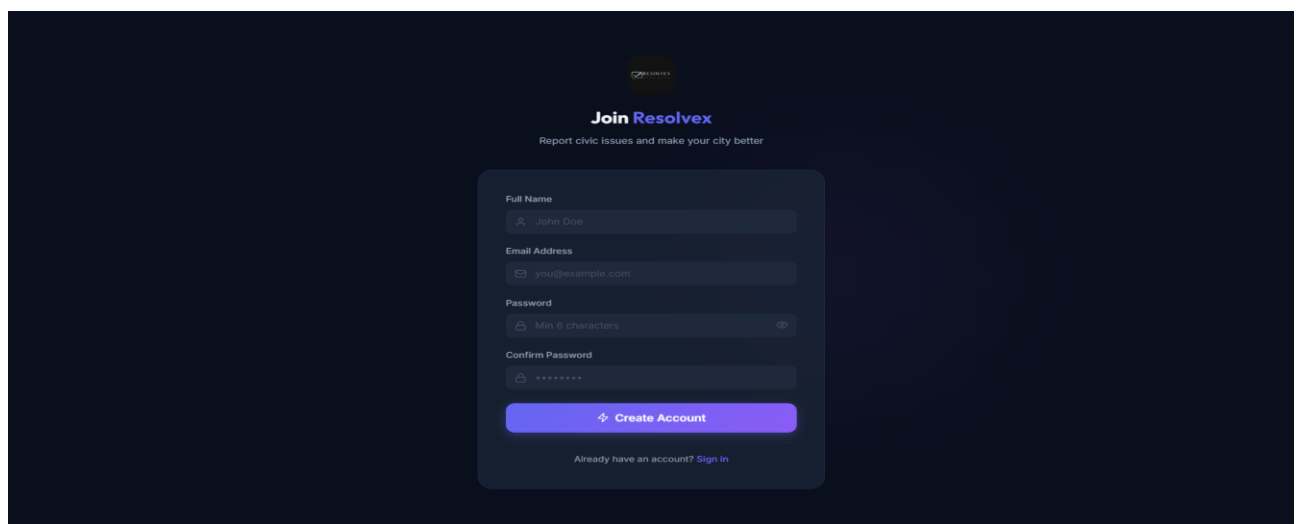


Fig 7.2 Register Page

- Here the User will be get register by filling the details as Name, Email, Password and confirm Password.

II. After that the verification is done of the user.

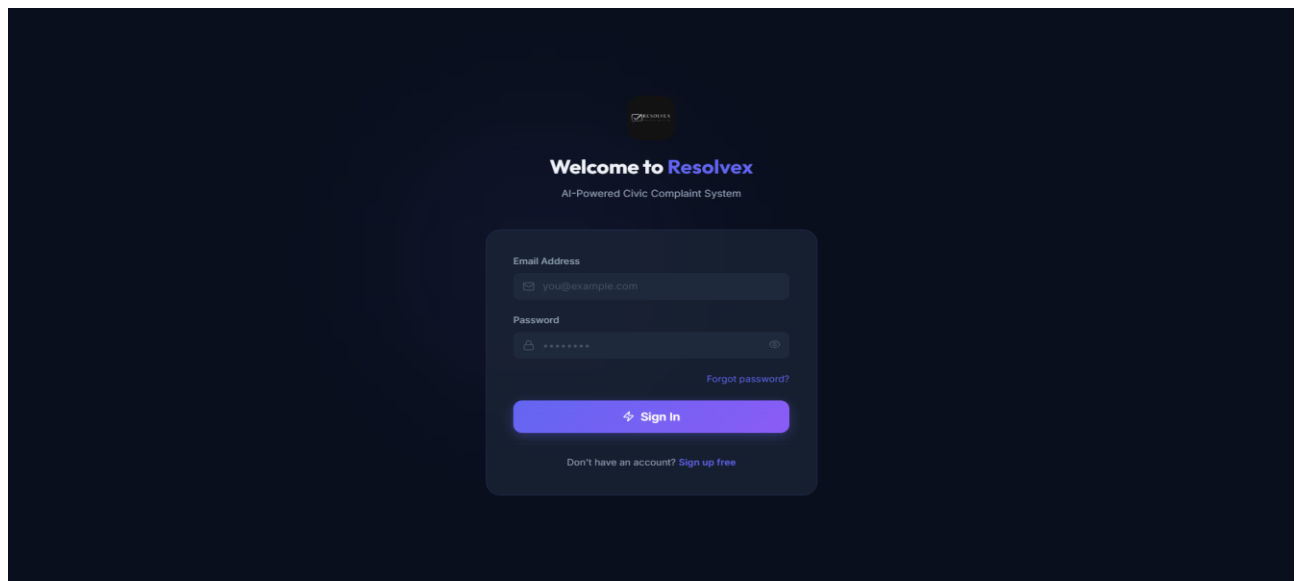


Fig 7.3 Login Page

- i. After the successfully register or verification the user now login from the login page.
- ii. By entering the email and password.
- iii. If the wrong details is entered it gives the error.

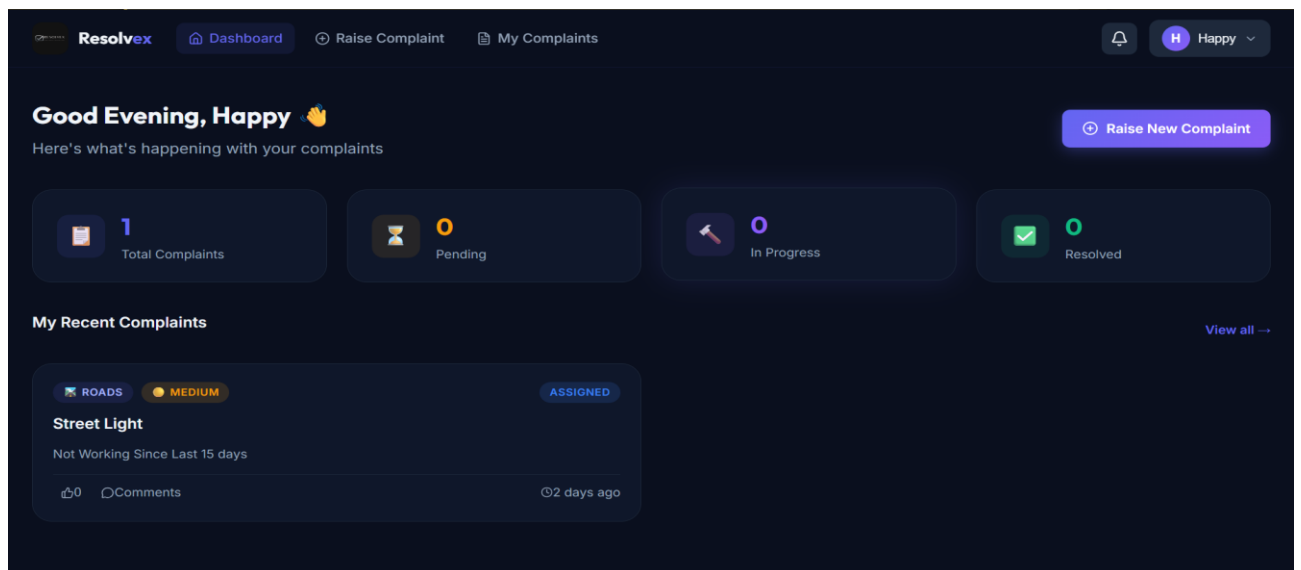


Fig 7.4 User Dashboard

- i. The User dashboard will be shown after the successful login where the user can see the complaint of it and its status.
- ii. Also the new complaint is raise from there.

ResolveX Dashboard Raise Complaint My Complaints

Raise a Complaint
Automatically analyze and prioritize your complaint.

1 Details 2 Location 3 Media 4 Review

Complaint Title *
Sewage Complaint

Description *
overflow

Analyze & Auto-Classify

Analysis Result Confidence: 85%
Suggested Category: **sanitation** Priority: **medium**
Identified key terms relating to sanitation department.
Suggestion: Please include specific street addresses or landmarks to help the sanitation team resolve this quickly.

Category
roads water electricity
sanitation public safety parks
noise animals other

Complaint Assistant
Support for filing
Hi! Need help filing your complaint? Tell me what issue you're facing.
How do I file a complaint? What issues can I report?
How long does resolution take? Can I track my complaint?
Ask ResolveXBot...

Fig 7.5 (a) Raise Complaint

- This is the First page of the complaint where the user will fill the details of the Complaint as title and the description.
- With the empty field it gives error to fill these required field.

ResolveX Dashboard Raise Complaint My Complaints

Raise a Complaint
Automatically analyze and prioritize your complaint.

1 Details 2 Location 3 Media 4 Review

Detect My Location

Address / Landmark
e.g. Near City Hall, Main Street Search

Map showing location with a pin. Labels: Devru, Sonipat. Legend: Leaflet, OpenStreetMap.

Click on the map to pin the exact location of the issue.

Back Next

Complaint Assistant
Support for filing
Hi! Need help filing your complaint? Tell me what issue you're facing.
How do I file a complaint? What issues can I report?
How long does resolution take? Can I track my complaint?
Ask ResolveXBot...

Fig 7.5 (b) Location

This is the second step of the Filing the complaint where the location of the problem is being provided.

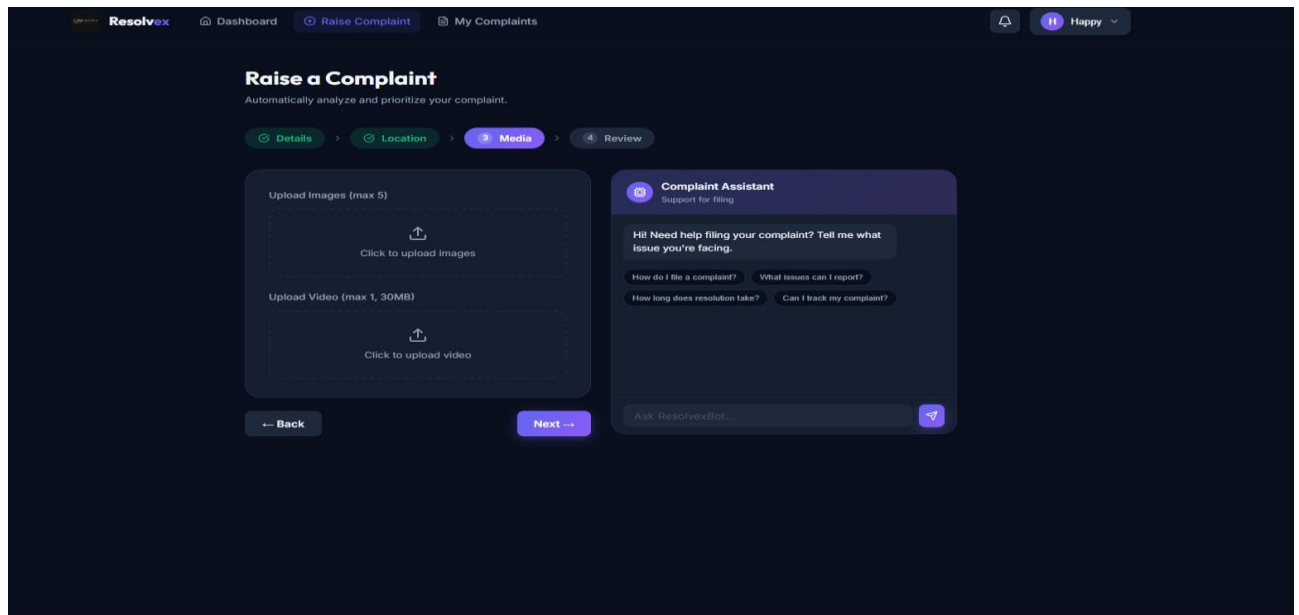


Fig 7.5 (c) Upload file

- i. Here the file of the related problem is being uploaded by the user as image or the video format.
- ii. It increase the chance of faster solving of the problem

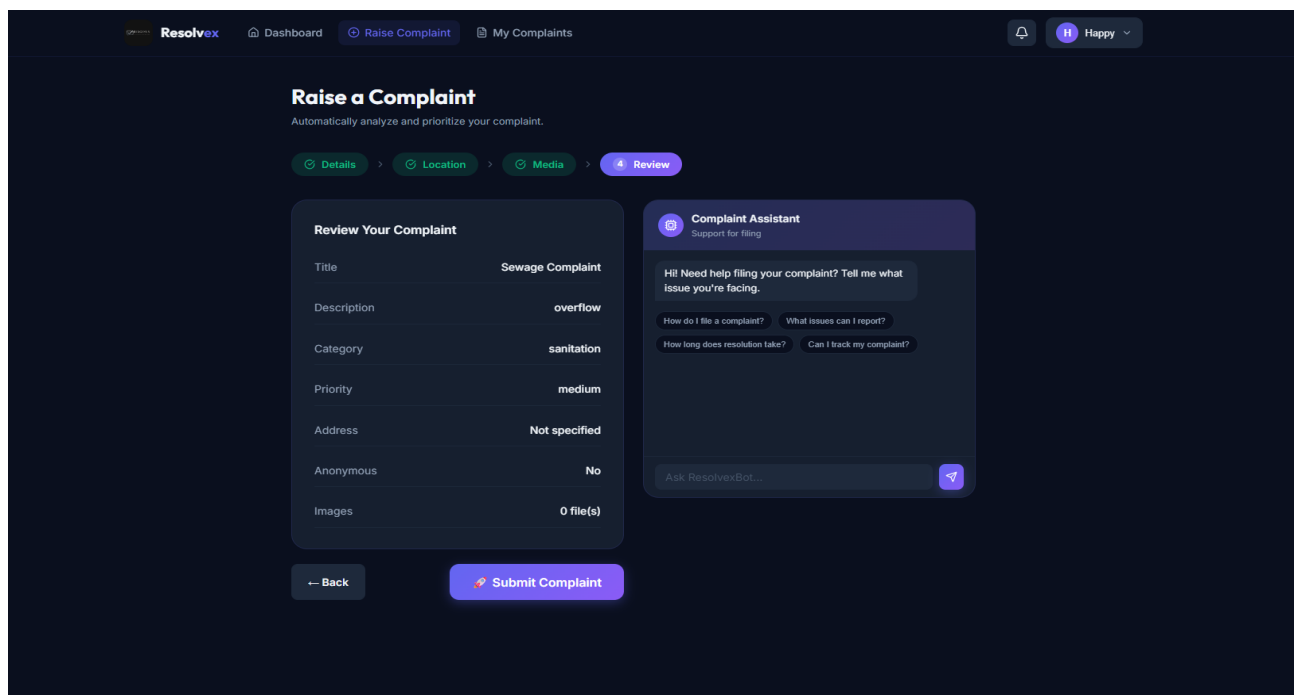


Fig 7.5 (d) Review before Submission

Before the final submission the User can review the Complaint for filling the correct data.

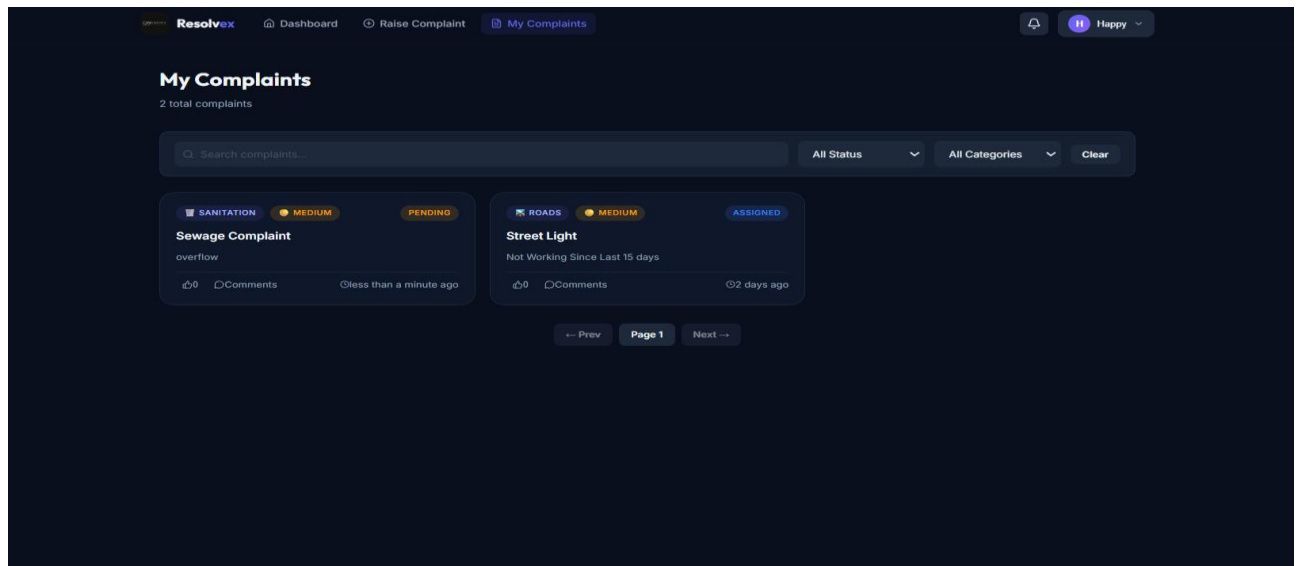


Fig 7.6 My Complaints

- i. This page shows the user which complaint he had made.
- ii. From here he can click on the any complaint and can see the details of the complaint with it status and dept assigned.

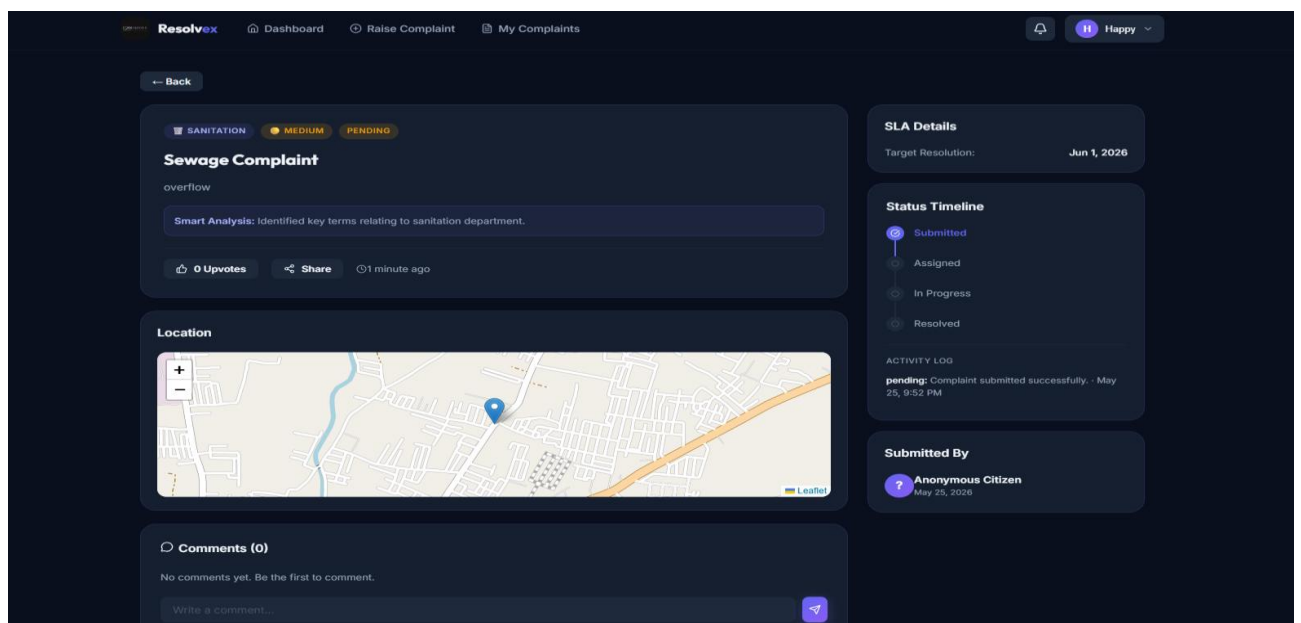
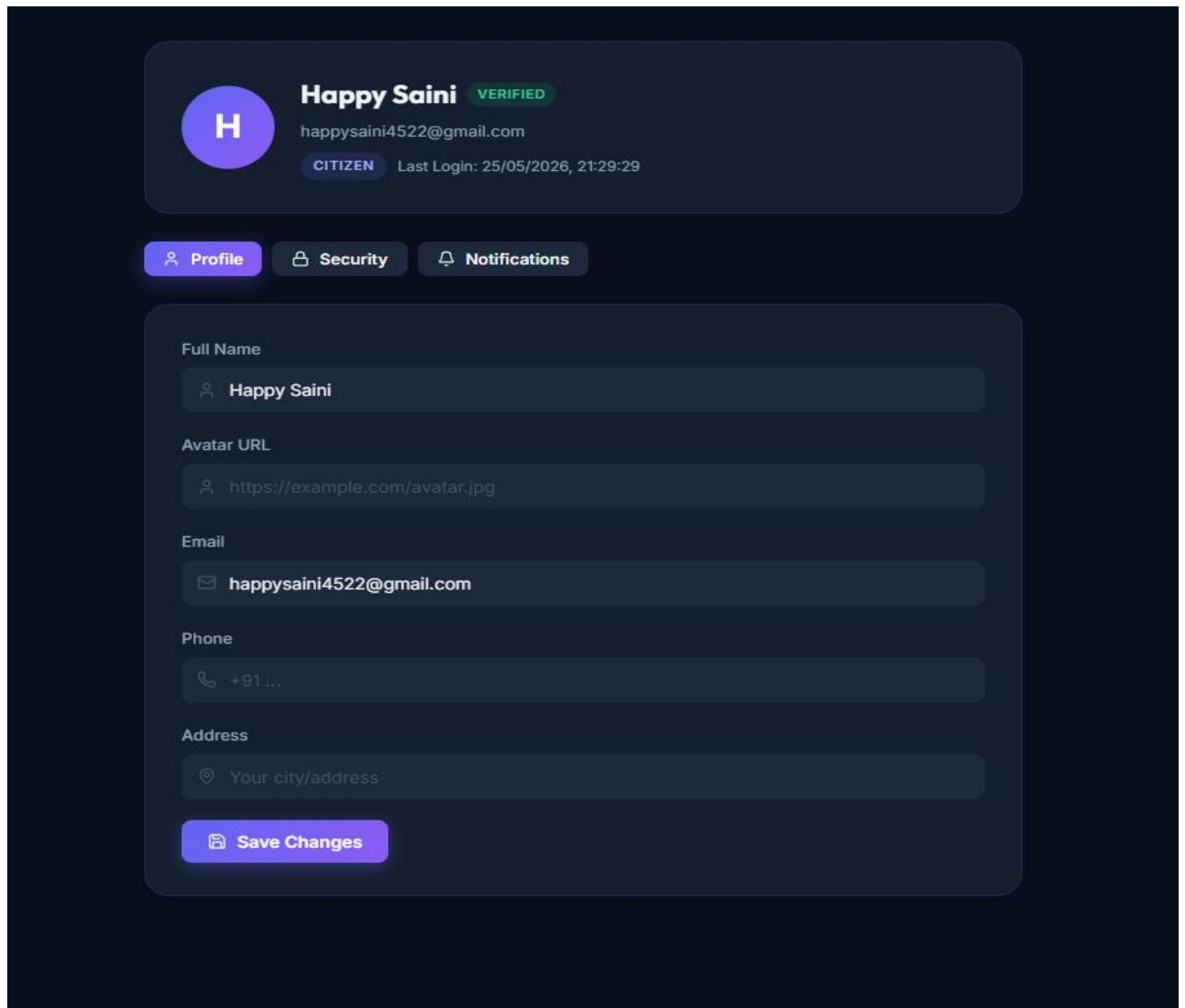


Fig 7.7 Complaint Status

This Screen shows the Details status or information of a particular complaint



The image shows a user profile interface on a dark background. At the top, there is a header section containing a circular profile picture with the letter 'H', the name 'Happy Saini' with a green 'VERIFIED' badge, the email 'happysaini4522@gmail.com', a 'CITIZEN' role badge, and the last login time '25/05/2026, 21:29:29'. Below this header are three tabs: 'Profile' (active), 'Security', and 'Notifications'. The main content area is a form with several input fields: 'Full Name' (containing 'Happy Saini'), 'Avatar URL' (containing 'https://example.com/avatar.jpg'), 'Email' (containing 'happysaini4522@gmail.com'), 'Phone' (containing '+91 ...'), and 'Address' (containing 'Your city/address'). A 'Save Changes' button is located at the bottom of the form.

Happy Saini VERIFIED
happysaini4522@gmail.com
CITIZEN Last Login: 25/05/2026, 21:29:29

Profile Security Notifications

Full Name
Happy Saini

Avatar URL
https://example.com/avatar.jpg

Email
happysaini4522@gmail.com

Phone
+91 ...

Address
Your city/address

Save Changes

Fig 7.8 User Profile

- i. From this page the user can update his Profile details.
- ii. As shown above .
- iii. And also the user can change it password from the security tab.

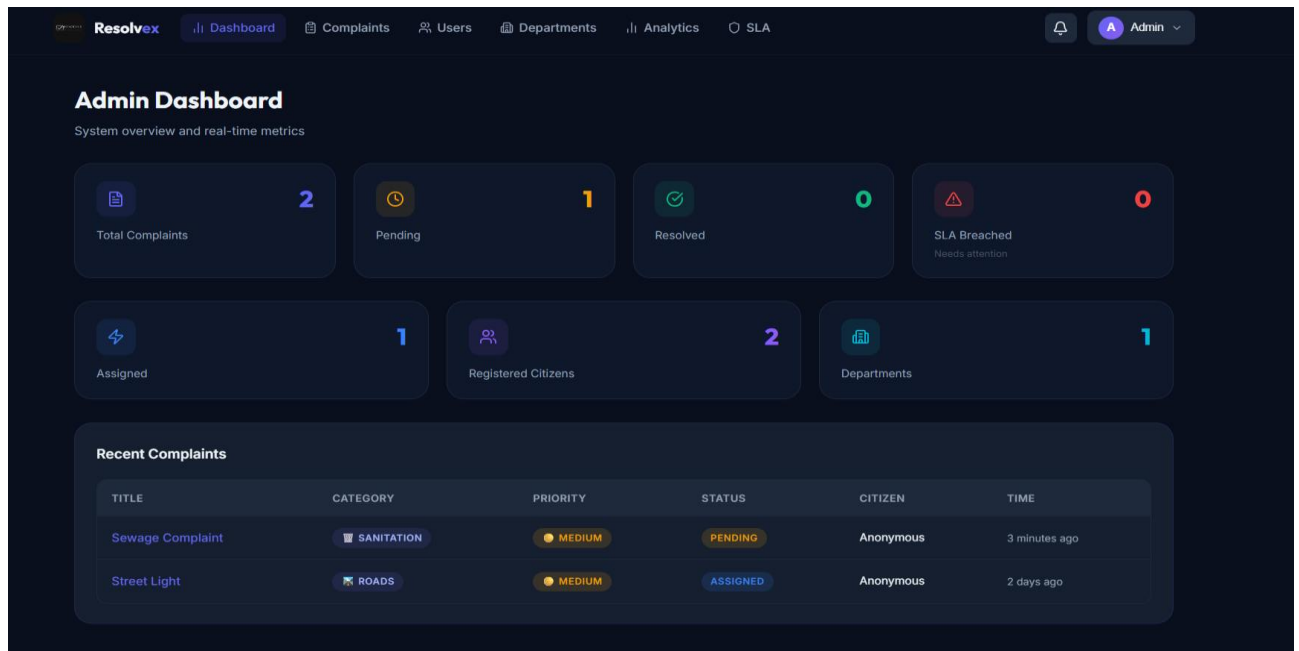


Fig 7.9 Admin Dashboard

The Above Screen shows the Admin Dashboard where you can see the many details such as:

Users, Complaints pending or all complaints, departments.

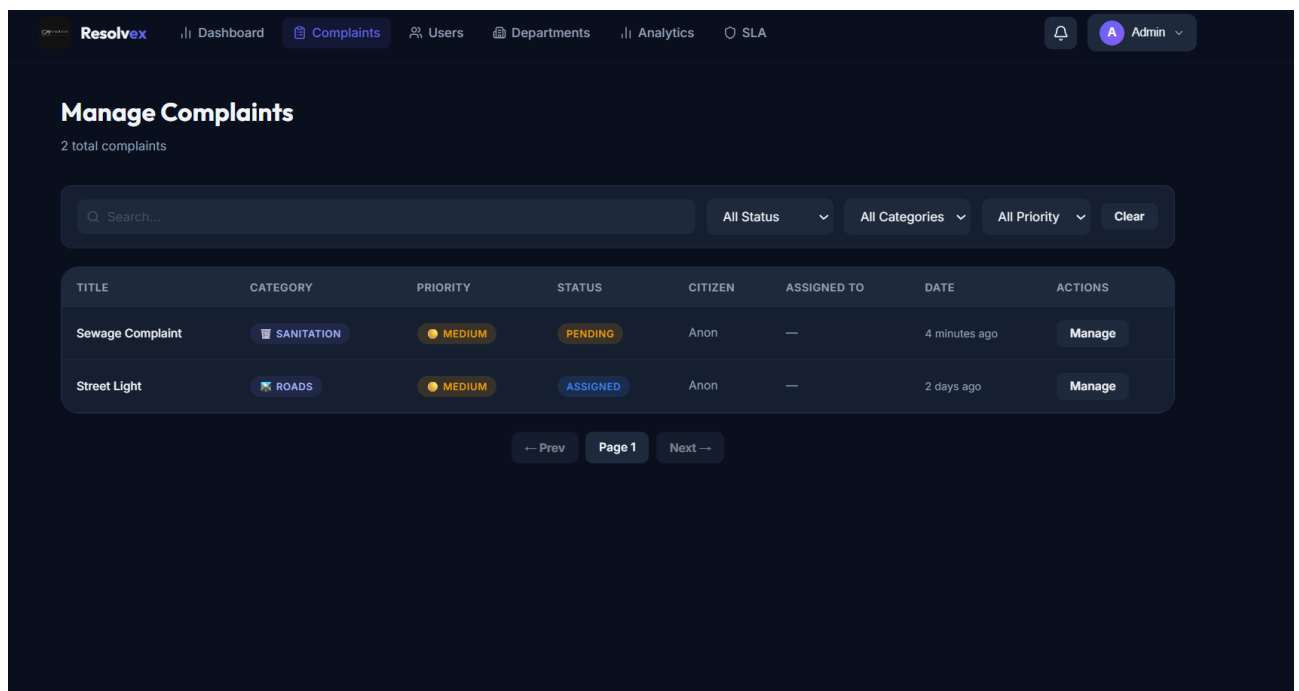


Fig 7.10 Admin Complaint Management

- From this page the admin can manage the complaint assigned the department or close the complaint from here.
- And do the many task on the complaint.

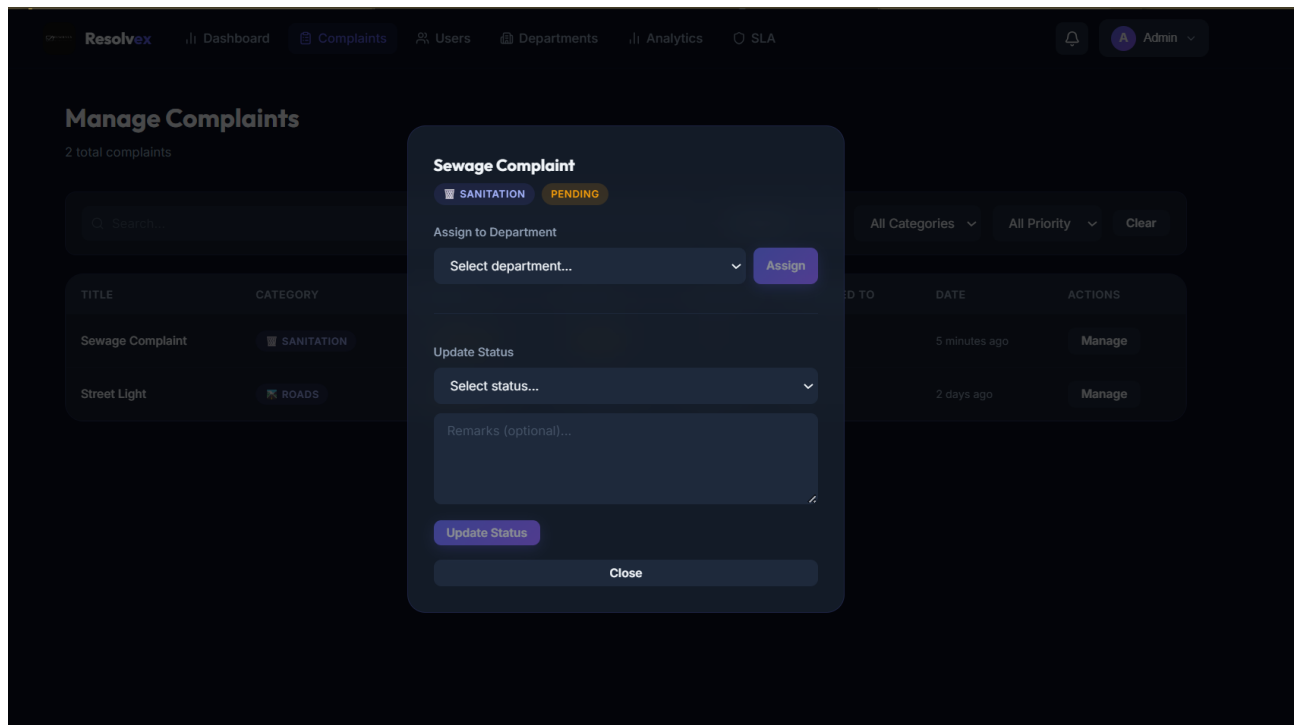


Fig7.11 Updating the status of the Complaint

As Shown above the user can update the status of the Complaint.

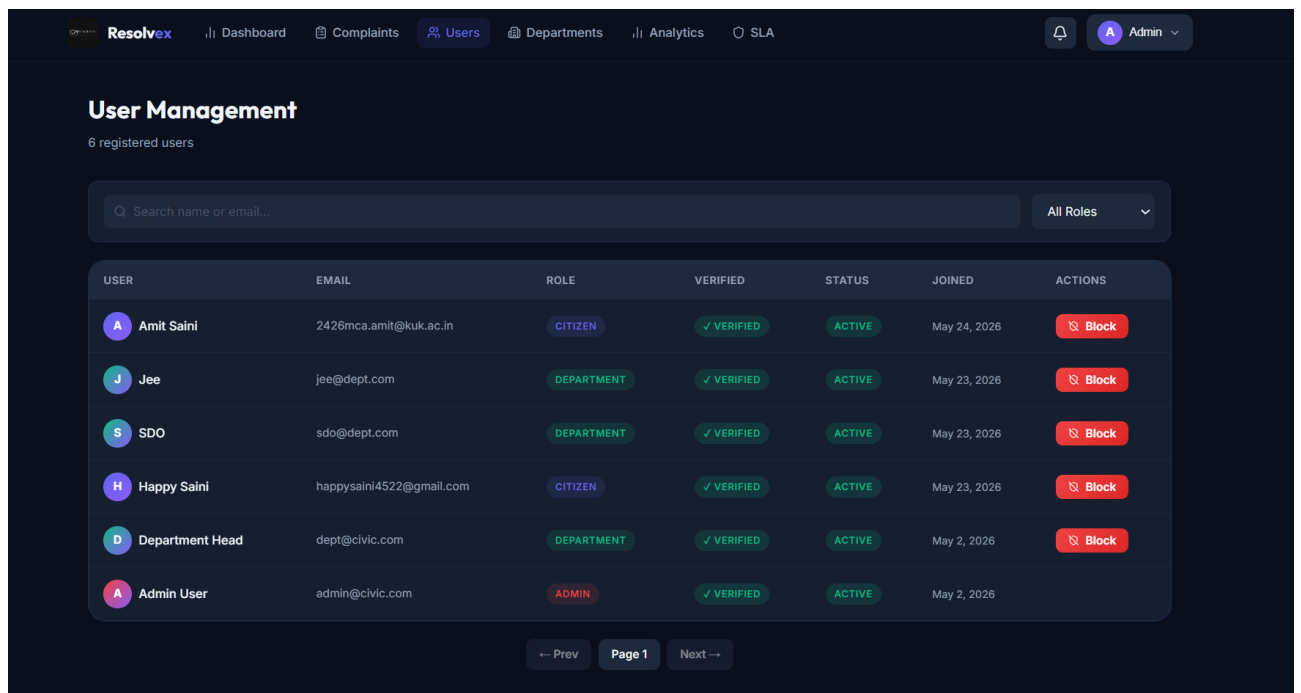


Fig 7.12 User Management by Admin

All the user whether citizen or department all can be seen by admin and can make changes in them.

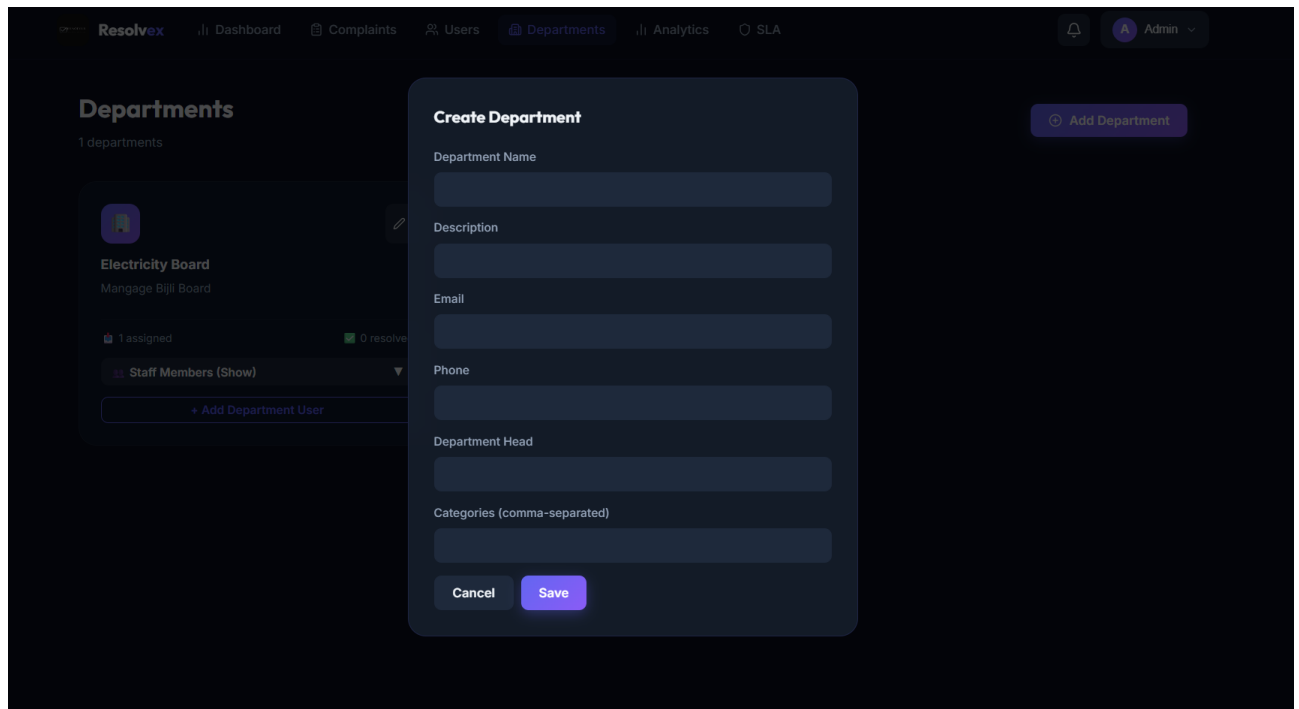


Fig 7.13 Managing the Department

- i. This page shows the how the Admin manage the department.
- ii. Adding the new department from the above form by filling the details as been asked.

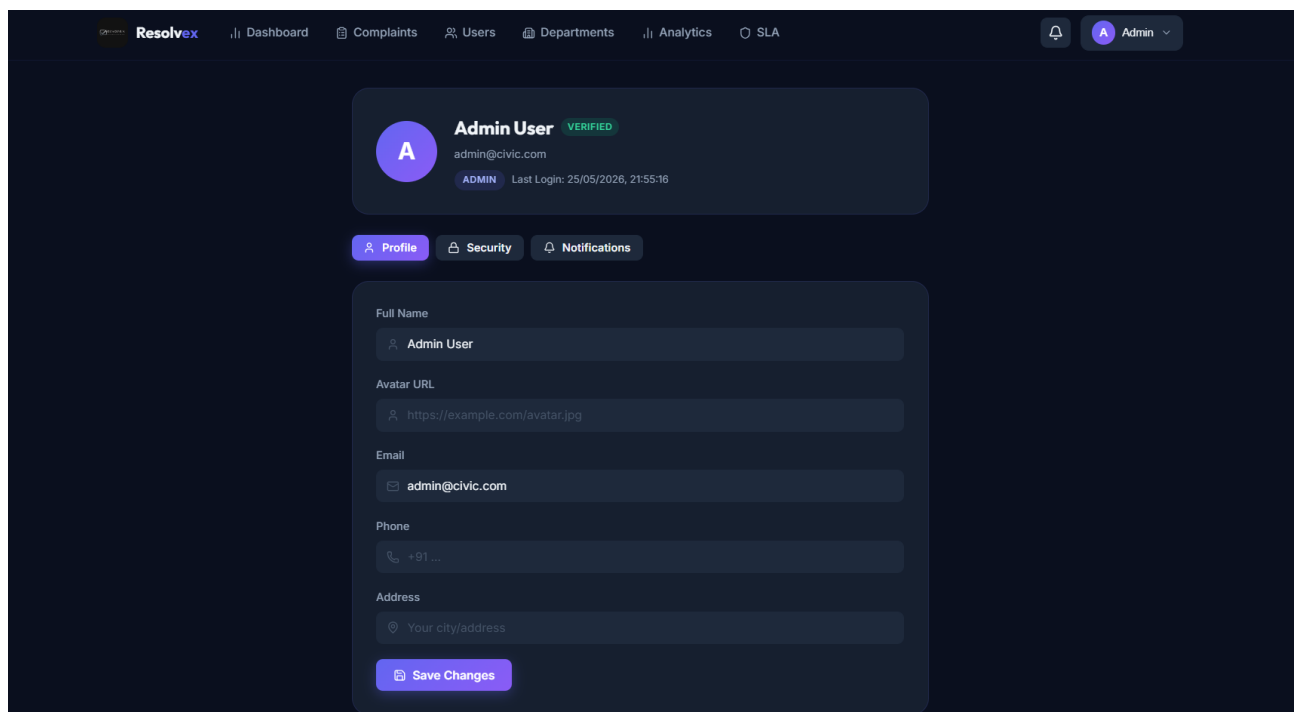


Fig 7.14 Admin Profile

The admin user can sees its profile also and can update it.

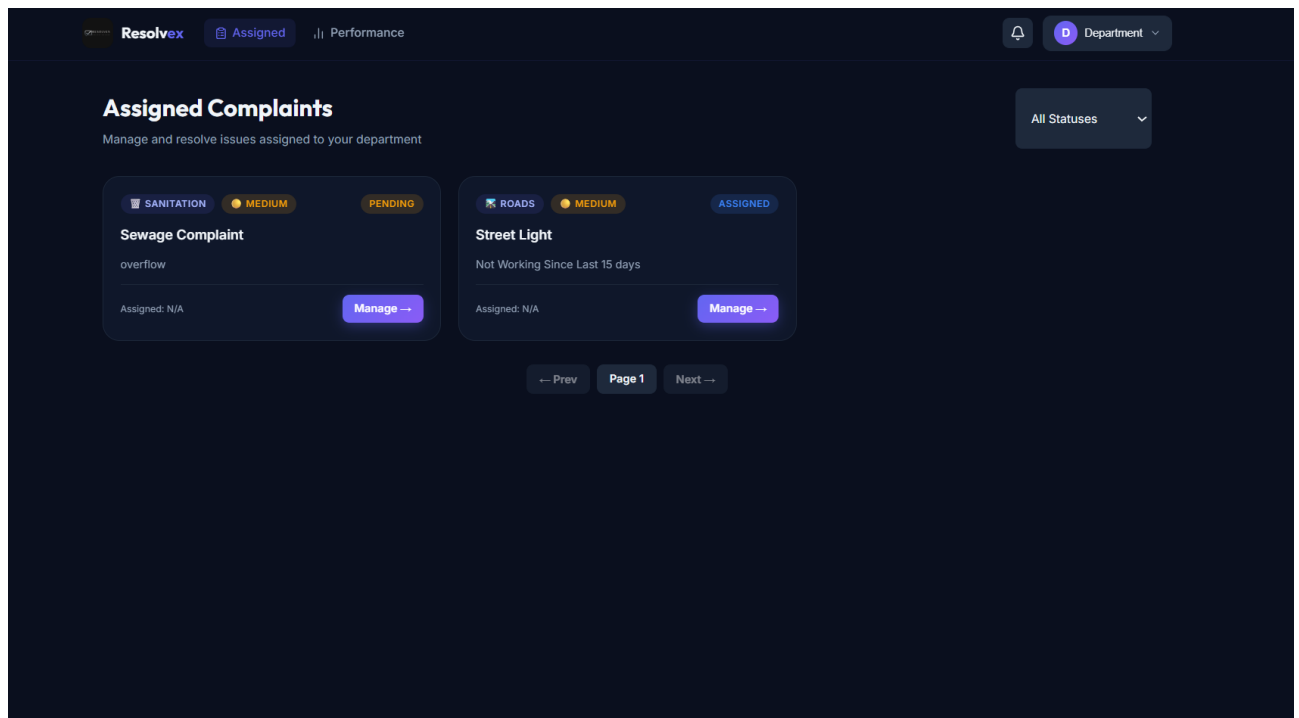


Fig 7.15 Complaint Assigned to Dept

- i. The department user can see the complaint assigned to it and start working on it.
- ii. The department can update the status on its end

Chapter 8

Limitation of the Project

1. Automated Testing - Limited Capability

The existing status of the Project is largely dependent on

- Manual Testing.
- API testing.
- UI testing.
- Authentication testing.

The system does not have established automated frameworks for performing the following test types:

- Unit Testing and unit testing automation.
- Integration Testing and integration testing automation.
- End-to-End Testing and end-to-end testing automation.
- Load Testing and load testing automation.

As a result, the following issues will occur:

- Some bugs may be undetected or hidden.
- Large scale testing is problematic.
- Regression testing takes a significant amount of time.

2. Internet Connectivity Dependency

The entire system is web-based and relies on access to the Internet. There are the following limitations:

- Citizens cannot access the system unless they have access to the Internet.
- The lack of reliable or poor Internet connectivity could impact the timely submission of complaints and notifications.

3. No Native Mobile Application

Current system available as web application only - No mobile version (Android or iOS)
- Limited user accessibility/No regional language support.

Many users speak native language - Problematic to process complaints in their native language; Very limited accessibility for users who do not speak English.

4. Basic Analytics Functionality

Basic analytics (complaint, SLA, department performance), but no advanced analytics; e.g. predictive analysis, future trends, resource allocation based on predictive reporting, and AI-based reporting.

5. Limited security measures

Current security measures include JWT, password hashing, role-based access control. There are currently no advanced security features; rate-limiting, two-factor authentication (2FA), advanced intrusion detection, web application firewall.

Chapter 9

Future Scope and Conclusion

9.1 Future Scope

The project's future scope will focus on the following main aspects: scalability, intelligence, accessibility, automated process, and smart city integration.

1. Mobile Application Development

The first major advance will be to create dedicated mobile application development for the following operating systems:

- iOS Device Application Development
- Android Device Application Development

2. GPS and Geolocation Integration

In the future, GPS and geolocation integration will include:

- Automatic GPS Location Tracking
- Live Mapping of Complaints
- Geo-Tagging
- Identification of Smart Regions

3. Advanced AI Integration

The AI modules can be developed further through:

- Machine learning methods
- Deep learning algorithms
- Natural language processing
- Predictive analytics
- Identification of fraudulent complaints

This will greatly improve the automation and efficiencies of the system.

4. Multilingual Support System

The future of the Support System will support multiple languages, benefiting the users of the system in many ways, such as:

- Increase Accessibility
- Increase User Participation
- More Rural Users

Also, AI translation methods can be added.

5. Smart City Integration

The Support System can be integrated with:

- Smart City Infrastructure
- Government Access Application Programming Interfaces (APIs)

- Geographic Information System (GIS) Systems
- Municipal Databases

6. IoT Integration

Internet of Things (IoT) Devices can be used to automate the detection of Civic Issues. Examples of IoT Devices would include:

- Smart Garbage Containers
- Water Leak Detection Sensors
- Pollution Sensors

These sensors can automatically create a complaint in the system.

7. Advanced Analytics Dashboard

Future advanced analytics will include:

- Predictive Analytics
- AI (Artificial Intelligence) generated reports
- Heat Map Intelligence
- Predictions of Departmental Efficiency
- Predictions of Complaint Trends

Thus, allowing administrators to make decisions based on data-driven analytics.

9.2 Conclusion

Resolving civic responsibility demonstrates how to set up a modern complaint management system for a city using an integrated approach with MERN architecture. It creates an efficient interaction model (system) to give citizens and local authorities greater communication, accountability, transparency through technology through improved workflow automation.

The main components of this new system include: Real-Time Communication, Cloud Media Management, AI Assisted Classification, and Secure Authenticated Access. These components enable the system to operate at an appropriate level of performance for deployment within a real-world environment.

This project also presents learning from the following areas: Modular Architecture, Scalable APIs, and Modern Front-End Technologies can be utilized to further extend the capabilities of the application across smart-city initiatives and large-scale municipal deployments.

In summary, the Resolvex project provides the public sector with a reliable, workable, usable, and scalable tool for managing civic incidents and improving the efficiency of public services for citizens in contemporary urban centres.

Chapter 10

Bibliography

- <https://nextjs.org/docs> – Next.js Official Documentation (for App Router, SSR, and project structure reference)
- <https://react.dev/> – React Official Documentation (for React 19 concepts, hooks, and component API reference)
- <https://nodejs.org/en/docs> – Node.js Official Documentation (for Node.js runtime API and usage)
- <https://expressjs.com/> – Express.js Official Documentation (for backend routing, middleware, and framework reference)
- <https://www.mongodb.com/docs/> – MongoDB Official Documentation (for database concepts, queries, and Mongoose integration)
- <https://mongoosejs.com/docs/> – Mongoose Official Documentation (for schema design, models, and ODM usage)
- <https://jwt.io/> – JWT.io (for JSON Web Token structure, understanding, and debugging)
- <https://tailwindcss.com/docs> – Tailwind CSS Official Documentation (for utility-first styling and class reference)
- <https://www.anthropic.com/api> – Anthropic Claude API Documentation (for AI content generation integration)
- <https://pollinations.ai/> – Pollinations AI (for AI-based image generation service reference)
- <https://helmetjs.github.io/> – Helmet.js Documentation (for HTTP security headers in Express)
- <https://chatgpt.com/> – ChatGPT (OpenAI) for assistance at various stages of development and concept clarification.
- <https://chat.deepseek.com/> – DeepSeek AI for guidance on UI design decisions and project-related discussions.
- <https://gemini.google.com/> - Used for making the flow charts and ER Diagram.